

Autoencoder

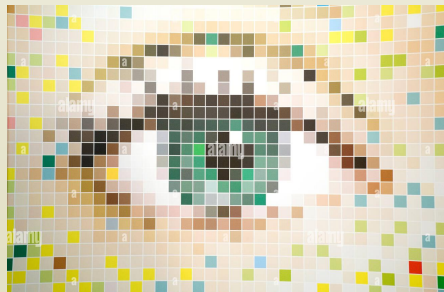
Join the lecture online on your dashboard.

Learning What Matters

A Socratic Pitch for Deep Autoencoders

Challenge of high-dimensionality

- Real world data is high-dimensional (image, audio, sensor data etc.).
- Useful information is much smaller.



→ $\approx 10^6$ dimensions

1024x1024

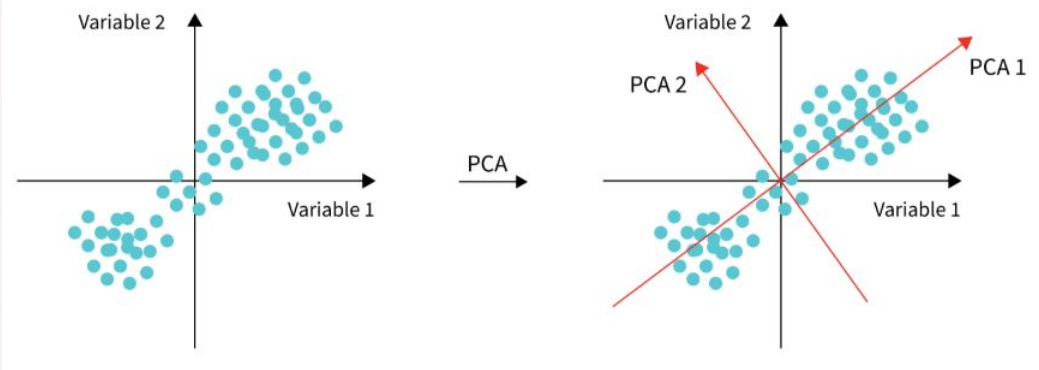
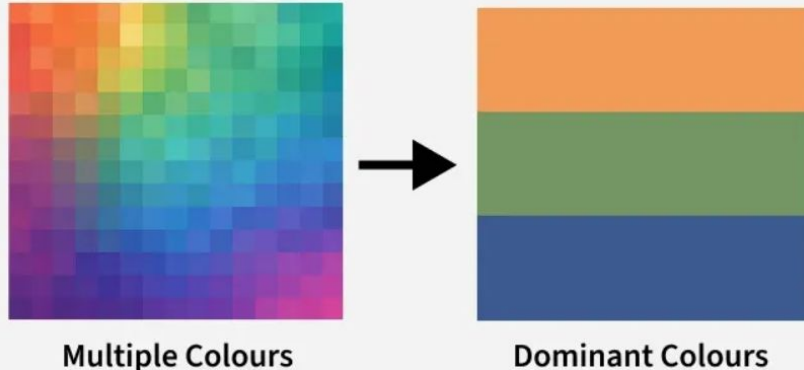
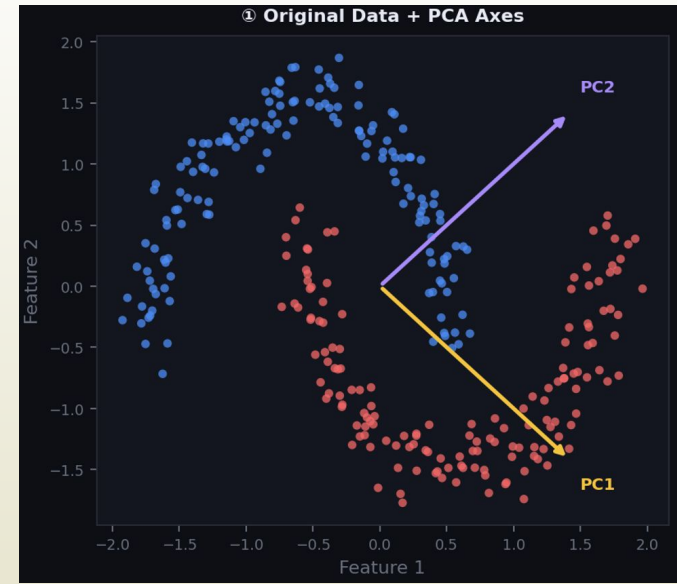
4



Can we compress the data without losing important signals?

But we had PCA?

- PCA is linear → Can't capture **complex non-linear** patterns.
- Real world data is often non-linear.



Autoencoders

- **What if a neural network could learn how to compress data automatically – and then reconstruct it?**

Autoencoders

Supervised → Unsupervised

- Autoencoders solves:
 - Dimensionality reduction for non-linear data
 - Noise removal
 - Feature learning
 - Data compression
 - Anomaly detection

Autoencoders

Supervised → Unsupervised

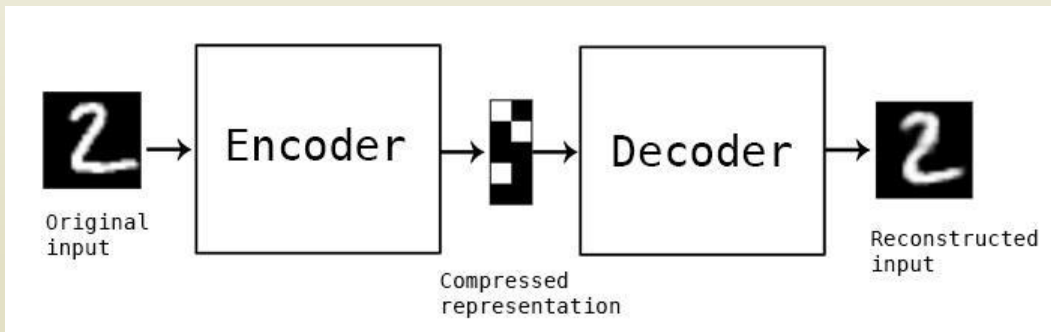
- Autoencoders solves:
 - Dimensionality reduction for non-linear data
 - Noise removal
 - Feature learning
 - Data compression
 - Anomaly detection

Autoencoders don't just reduce data, they learn what matters.

Autoencoders

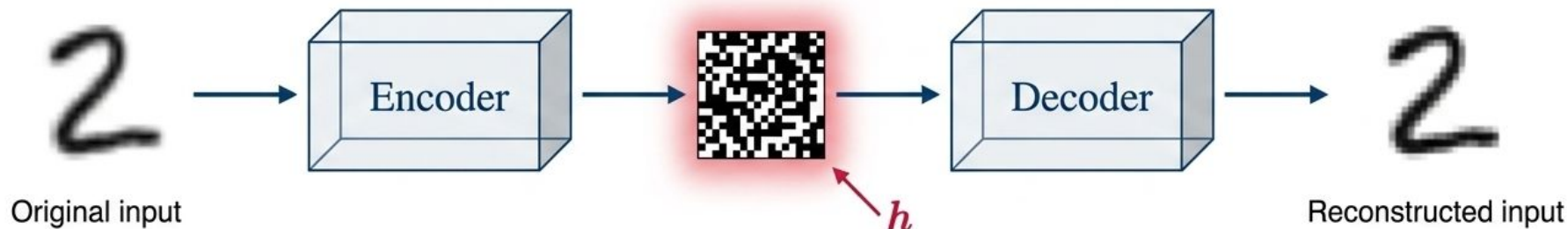
Supervised $\rightarrow$ Unsupervised

- Autoencoders transform unsupervised task to supervised task.
- Use inputs as the ground truth labels.



The Meaning of ' h ': The Latent Space

If the neural network perfectly reconstructs $\hat{x}_i$, what does that indicate about h ?

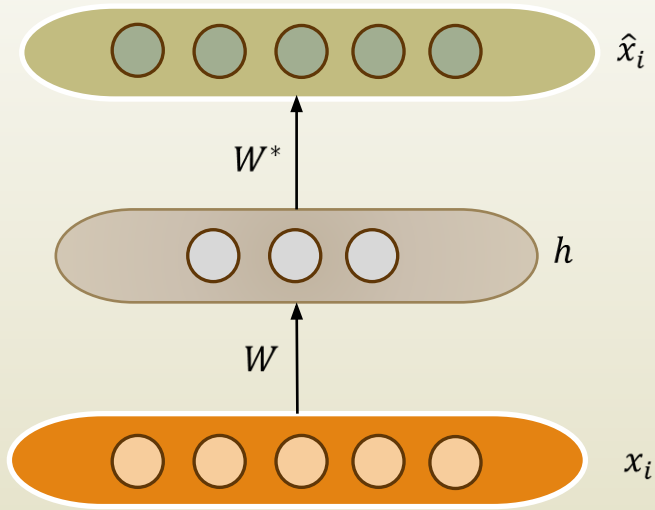


1. Let $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$.
2. If reconstruction is perfect, $\mathbf{h}$ must be a loss-less representation of $\mathbf{x}_i$.
3. Therefore, $\mathbf{h}$ successfully represents all important underlying characteristics of $\mathbf{x}_i$.

**Insight: Autoencoders don't just reduce data;
they act as automated feature extractors.**

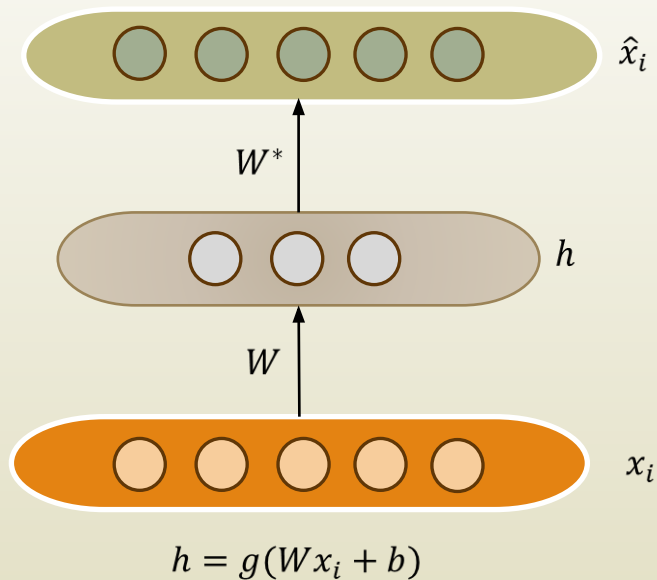
I: Under-Complete Autoencoders

Autoencoders



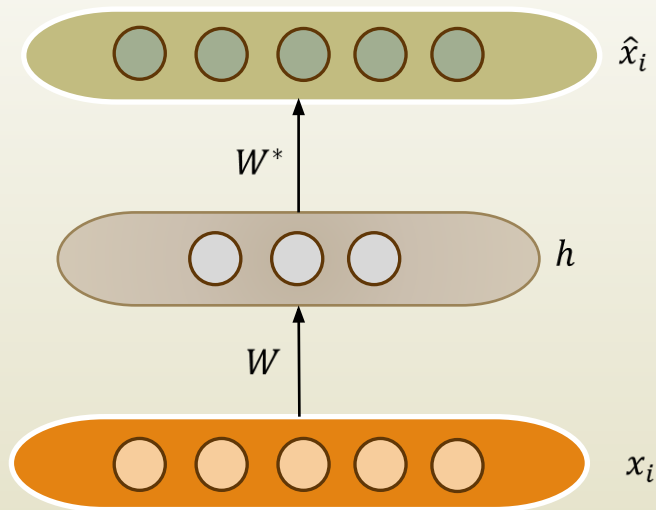
- A special type of feedforward neural network.

Autoencoders



- A special type of feedforward neural network.
- **Encodes** its input x_i to a hidden representation h .

Autoencoders

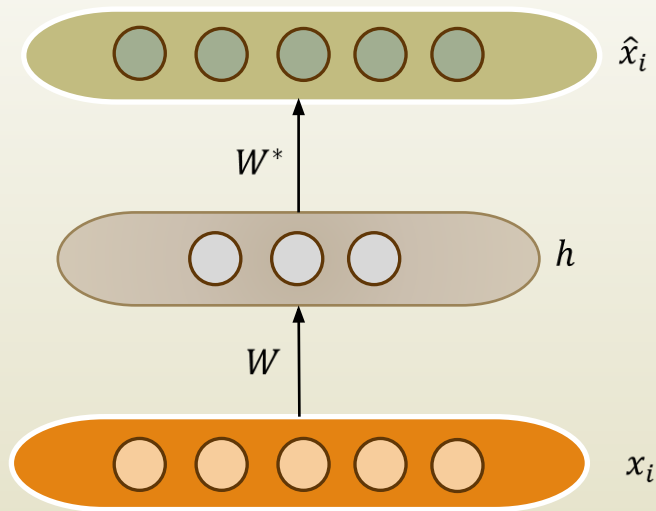


$$h = g(Wx_i + b)$$

$$\hat{x}_i = f(W^*h + c)$$

- A special type of feedforward neural network.
- **Encodes** its input x_i to a hidden representation h .
- **Decodes** the hidden representation h into the input.
- **Objective:** Make $\hat{x}_i$ closer to x_i .

Autoencoders

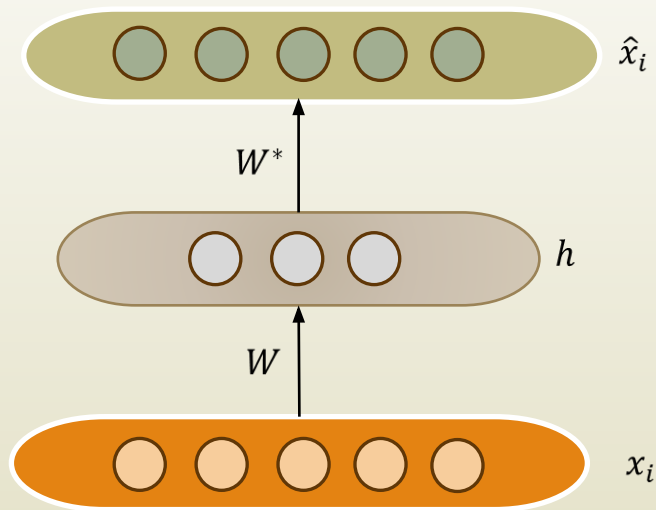


$$h = g(Wx_i + b)$$

$$\hat{x}_i = f(W^*h + c)$$

- Let $\dim(h) < \dim(x_i)$.
- If the NN is able to reconstruct $\hat{x}_i$ perfectly, what does it indicate about $\mathbf{h}$?

Autoencoders

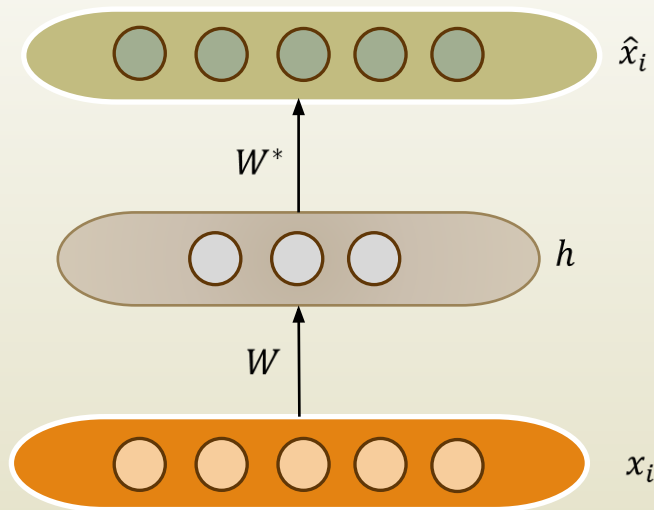


$$h = g(Wx_i + b)$$

$$\hat{x}_i = f(W^*h + c)$$

- Let $\dim(h) < \dim(x_i)$.
- If the NN is able to reconstruct $\hat{x}_i$ perfectly, what does it indicate about $\mathbf{h}$?
- h is a loss-less representation of x_i .
- OR h represents all important characteristics of x_i .

Autoencoders



$$h = g(Wx_i + b)$$

$$\hat{x}_i = f(W^*h + c)$$

- Let $\dim(h) < \dim(x_i)$.
- If the NN is able to reconstruct $\hat{x}_i$ perfectly, what does it indicate about **h** ?
- h is a loss-less representation of x_i .
- OR h represents all important characteristics of x_i .

An autoencoder where $\dim(h) < \dim(x_i)$ is called an **under complete autoencoder**.

<https://introduction-to-autoencoders.vercel.app/>

A. Pick Dataset

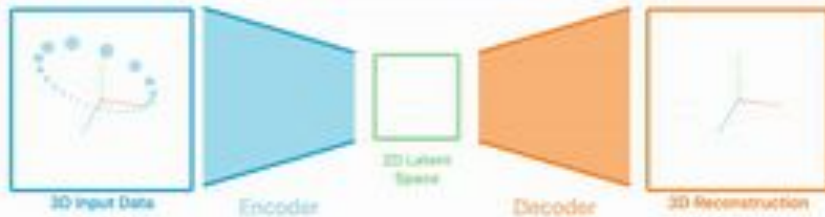


B. Train Autoencoder

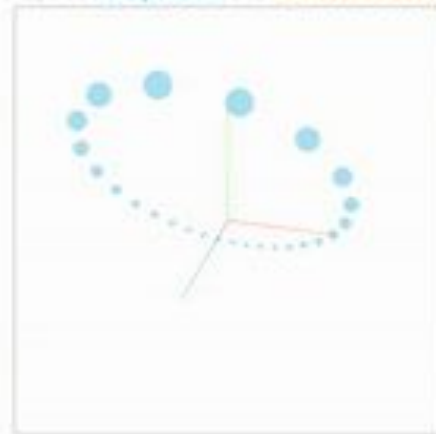


Epochs
000,000

Loss
...



Layered 3D input Data and 3D Reconstruction



2D Latent Space with Opposite Gradients ✓

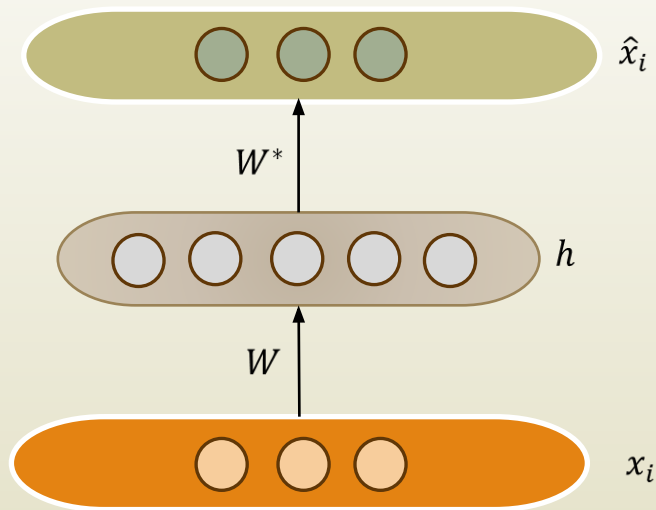


**What happens if the bottleneck
is accidentally larger than the
original input space?**

$$\dim(h) \geq \dim(x)$$

II: Over-Complete Autoencoders

Autoencoders

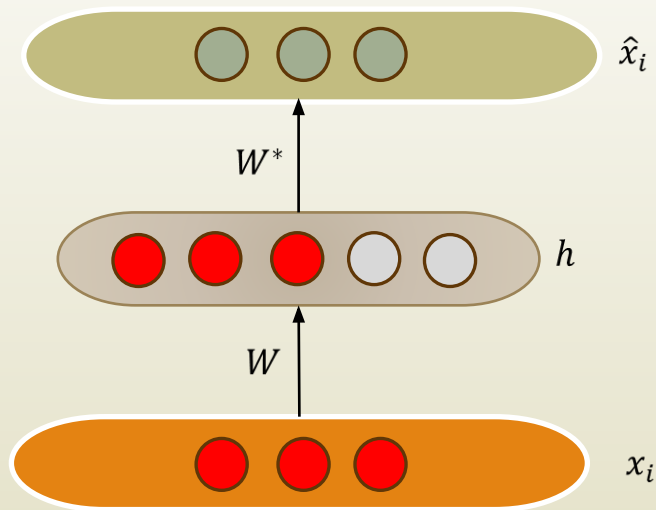


$$h = g(Wx_i + b)$$

$$\hat{x}_i = f(W^*h + c)$$

- Let $\dim(h) \geq \dim(x_i)$.
- The NN can always learn a trivial encoding by just copying:
 - x_i to h
 - h to $\hat{x}_i$

Autoencoders

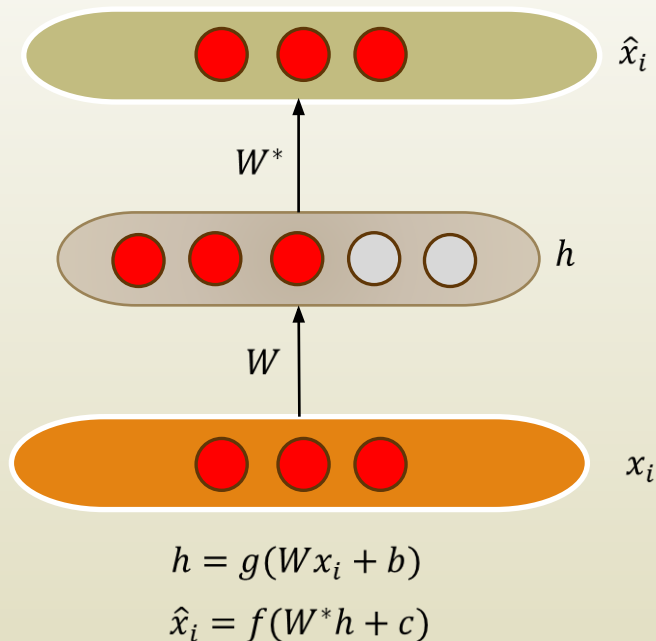


$$h = g(Wx_i + b)$$

$$\hat{x}_i = f(W^*h + c)$$

- Let $\dim(h) \geq \dim(x_i)$.
- The NN can always learn a trivial encoding by just copying:
 - x_i to h
 - h to $\hat{x}_i$

Autoencoders



- Let $\dim(h) \geq \dim(x_i)$.
- The NN can always learn a trivial encoding by just copying:
 - x_i to h
 - h to $\hat{x}_i$
- NN is not learning anything about data → **Useless encoding**

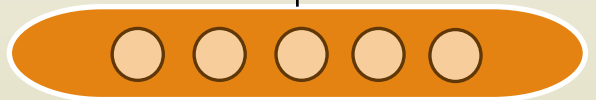
An autoencoder where $\dim(h) \geq \dim(x_i)$ is called an **over complete autoencoder**.

Activations f and g

$$\hat{x}_i = f(W^*h + c)$$

 W^*

$$h = g(Wx_i + b)$$

 W  x_i

0 1 1 0 0

Binary
inputs

- **Binary inputs**

- Which function is the most appropriate for decoder?

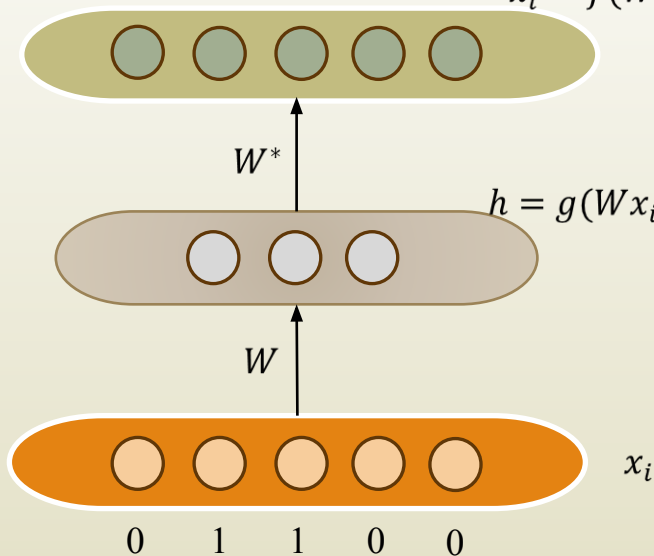
$$\hat{x} = \tanh(W^*h + c)$$

$$\hat{x} = W^*h + c$$

$$\hat{x} = \text{logistic}(W^*h + c)$$

Activations f and g

$$\hat{x}_i = f(W^*h + c)$$



Binary
inputs

g is typically a sigmoid function.

- **Binary inputs**

- Which function is the most appropriate for decoder (f)?

$$\hat{x} = \tanh(W^*h + c)$$

$$\hat{x} = W^*h + c$$

$$\hat{x} = \text{logistic}(W^*h + c)$$

- **Logistic**

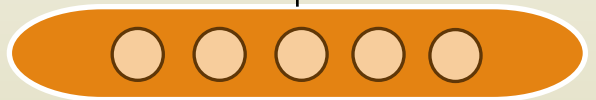
- It naturally restricts all inputs to be between 0 & 1.

Activations f and g

$$\hat{x}_i = f(W^*h + c)$$

 W^*

$$h = g(Wx_i + b)$$

 W  x_i

0.	1.	1.	0.	5
1	5	1	2	

Real-valued
inputs

- **Real-valued inputs**

- Which function is the most appropriate for decoder (f)?

$$\hat{x} = \tanh(W^*h + c)$$

$$\hat{x} = W^*h + c$$

$$\hat{x} = \text{logistic}(W^*h + c)$$

Activations f and g

$$\hat{x}_i = f(W^*h + c)$$

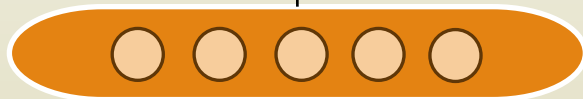


W^*

$$h = g(Wx_i + b)$$



W



x_i

0. 1. 1. 0. 5
1 5 1 2

Real-valued
inputs

g is typically a sigmoid function.

- Real-valued inputs

- Which function is the most appropriate for decoder (f)?

$$\hat{x} = \tanh(W^*h + c)$$

$$\hat{x} = W^*h + c$$

$$\hat{x} = \text{logistic}(W^*h + c)$$

- $\hat{x} = W^*h + c$

- Because *logistic* and *tanh* will restrict the values.

Different data types require completely different reconstruction rules. How do we configure the decoder and measure its loss?

Loss Functions: Fitting Output to Data

Continuous / Real-Valued Inputs

- Decoder Activation: Linear or continuous (f)
- Metric: Calculates the geometric distance between points.

$$\mathcal{L}(\theta) = (\hat{x}_i - x_i)^T (\hat{x}_i - x_i)$$

(Squared Error)

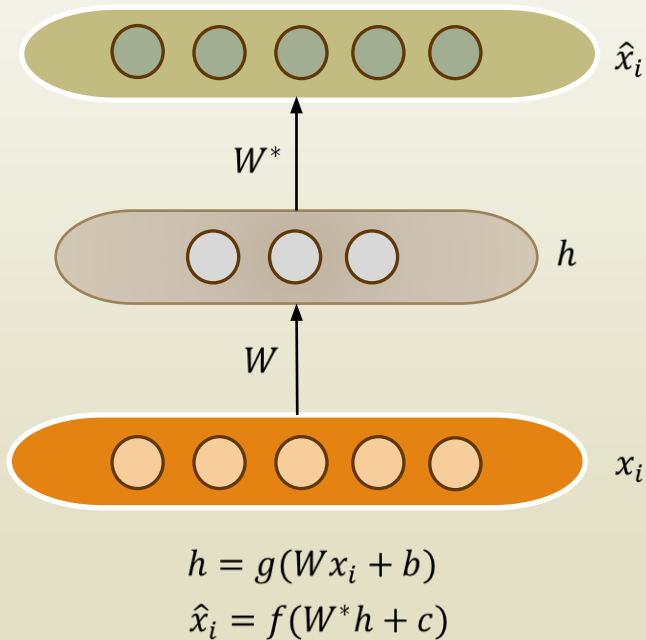
Binary / Logistic Inputs

- e.g., input: [0 1 1 0 1], output probabilities: [0.2 0.75 0.9 0.11 0.8]
- Decoder Activation: Logistic units.
 $\hat{x}_i = \text{logistic}(W^*h + c)$
- Cross-entropy proves superior to squared error.

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$

Loss function

For real-valued inputs



Objective: Reconstruct $\hat{x}_i$ as close to x_i as possible.

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

$m = \text{no. of samples}, n = \text{dimensions}$

In matrix form:

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

Train the autoencoder using backpropagation.

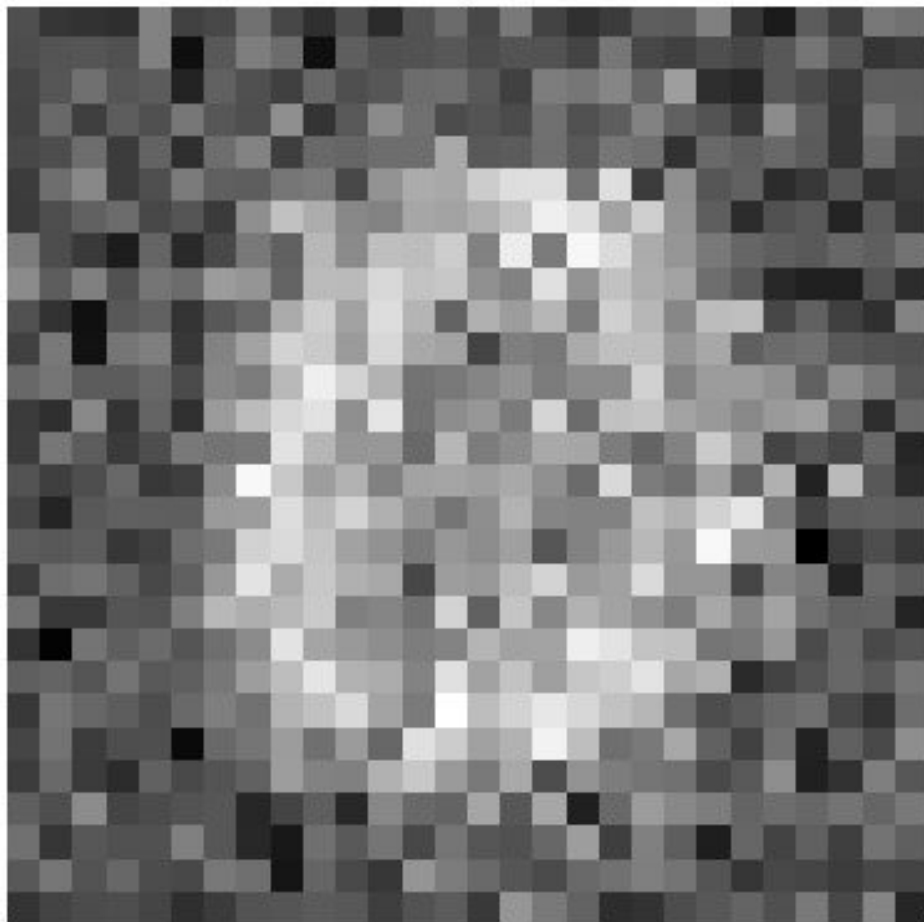
The Activation & Loss Decision Matrix

The objective is always to reconstruct predicted output as close to original input as possible.

Input Type	Decoder Activation (f)	Loss Function (L)
Real-Valued Inputs	Linear Activation Because logistic/tanh inappropriately restrict values.	Squared Error Loss $L(\theta) = (\hat{x}_i - x_i)^T (\hat{x}_i - x_i)$
Binary Inputs	Logistic / Sigmoid Activation Restricting outputs strictly between 0 and 1 as probabilities.	Cross-Entropy Loss $L(\theta) = - \sum (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$

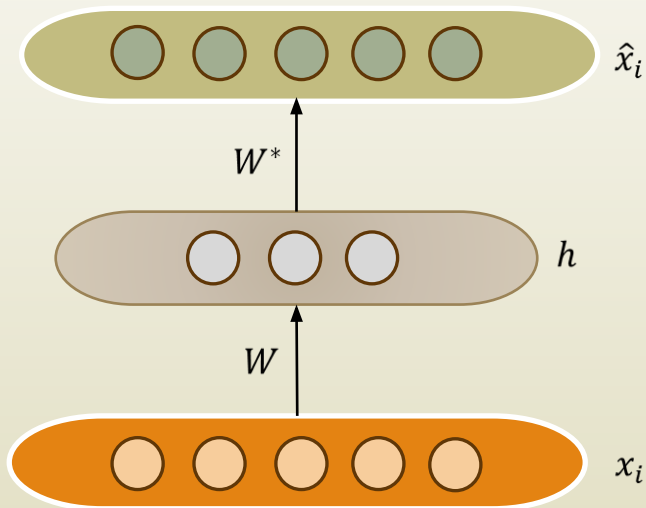
Training

Step 10



Loss function

For real-valued inputs



$$h = g(Wx_i + b)$$
$$\hat{x}_i = f(W^*h + c)$$

We need to calculate:

$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*}$$

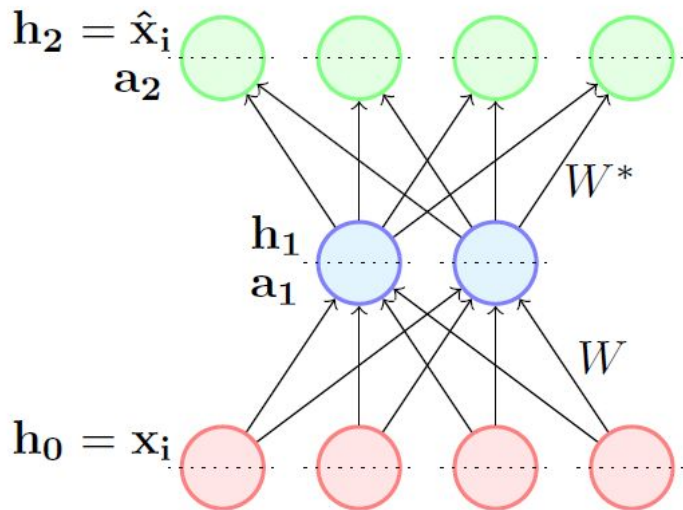
and

$$\frac{\partial \mathcal{L}(\theta)}{\partial W}$$

(Ignoring biases for now)

The gradients

$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



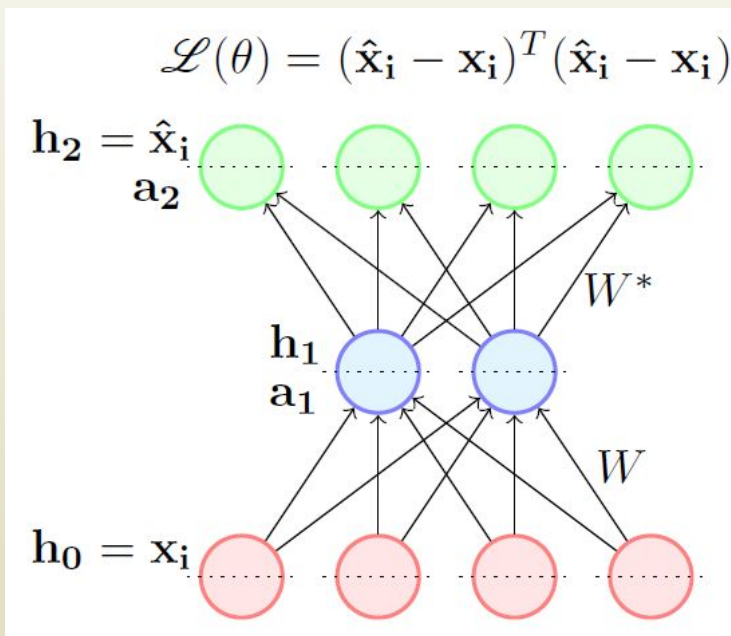
$$\mathcal{L} \leftarrow h_2 \leftarrow a_2 \leftarrow W^*$$

$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial W^*}$$

Similarly

$$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial W}$$

The gradients



$$\mathcal{L} \leftarrow h_2 \leftarrow a_2 \leftarrow W^*$$

$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial W^*}$$

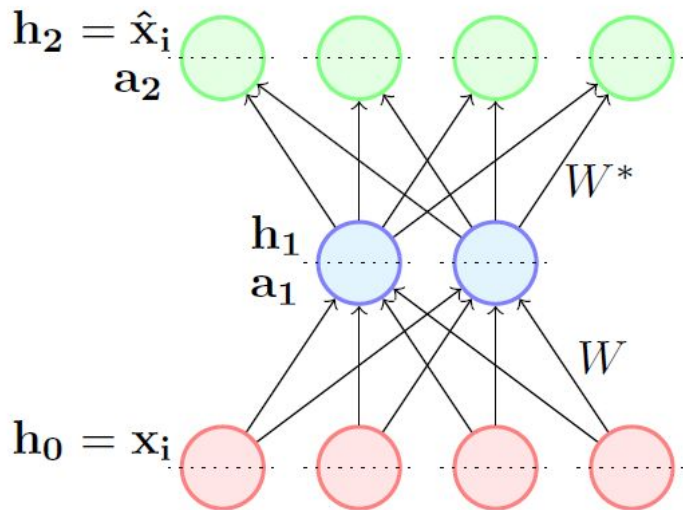
Similarly

$$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial W}$$

Same as earlier
backpropagation
examples.

The gradients

$$\mathcal{L}(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$



$$\frac{\partial \mathcal{L}(\theta)}{\partial h_2} = \frac{\partial \mathcal{L}(\theta)}{\partial \hat{x}_i} = \nabla_{\hat{x}_i} \{(\hat{x}_i - x_i)^T (\hat{x}_i - x_i)\}$$

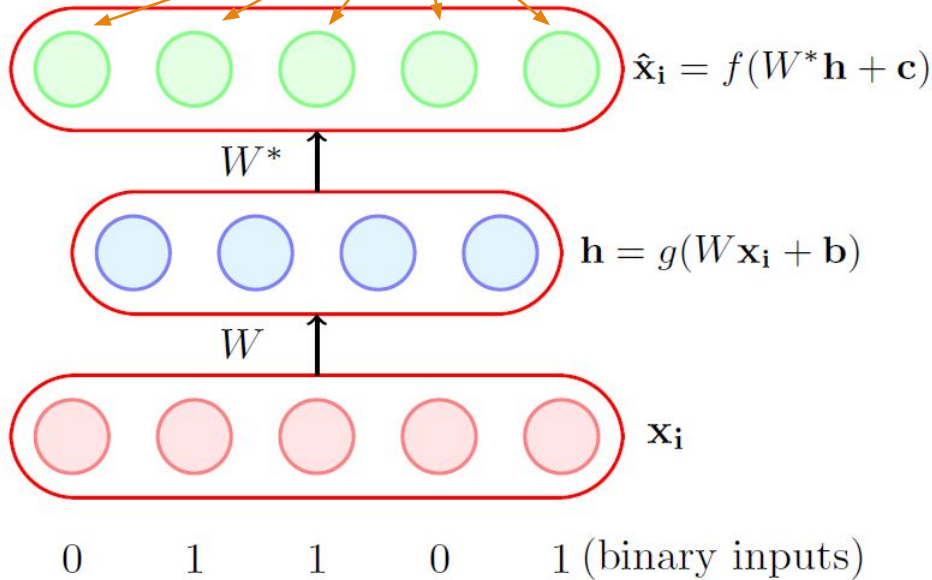
$$[\because \nabla_u (u^T u) = 2u]$$

$$= 2(\hat{x}_i - x_i).I$$

$$= 2(\hat{x}_i - x_i)$$

The gradients

Logistic
units

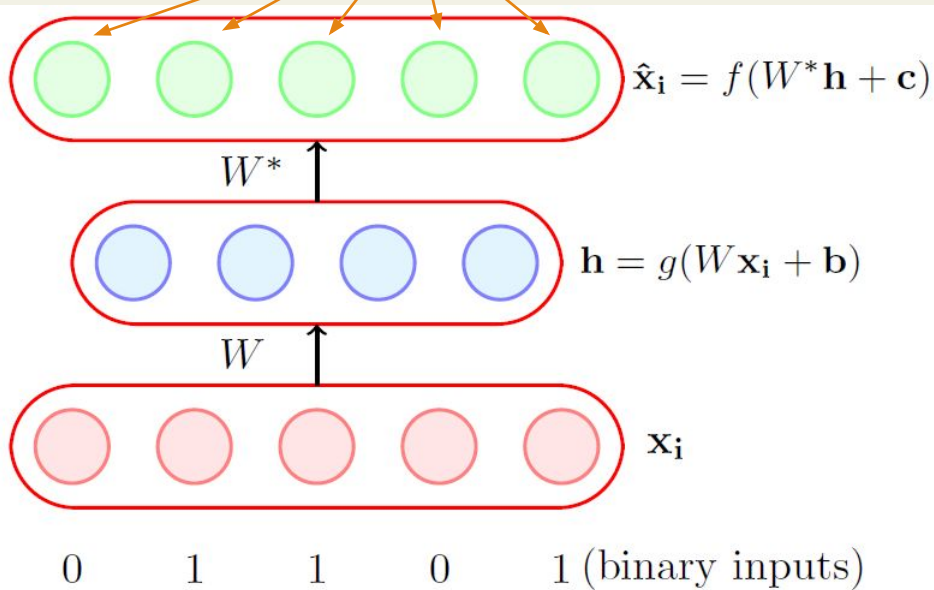


For binary inputs

- In the output layer, we have logistic units, because we have to reconstruct binary values.
- *Preactivation $a = W^*h + c$*
- *Activation function – Logistic*
- *Output – $\hat{x}_i$ (between 0 & 1)*

The gradients

Logistic
units

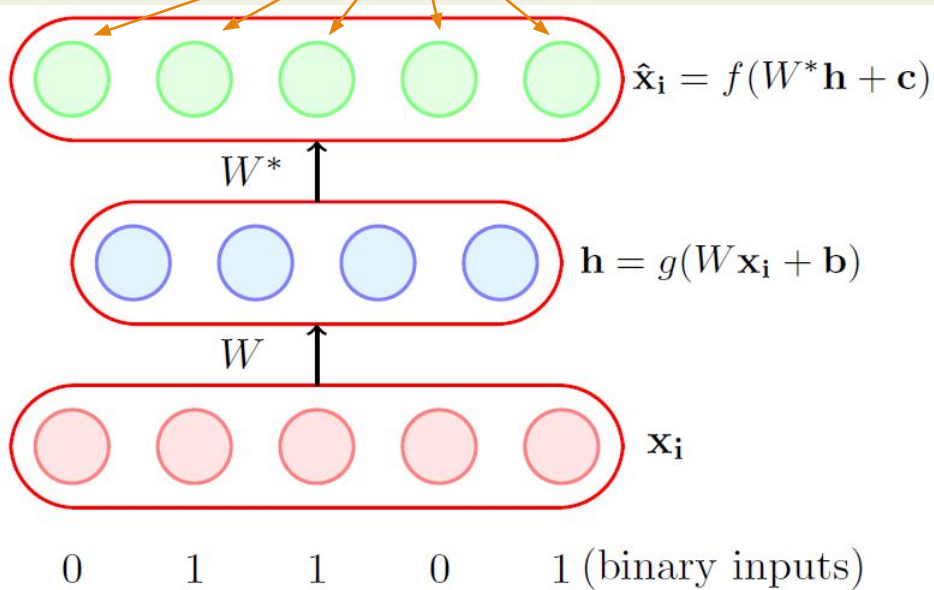


For binary inputs

- In the output layer, we have logistic units, because we have to reconstruct binary values.
- Sample input: [0 1 1 0 1]
- Sample output: [0.2 0.75 0.9 0.11 0.8]
(can be interpreted as probabilities)
- **Can we use the squared error like in previous case?**

The gradients

Logistic
units

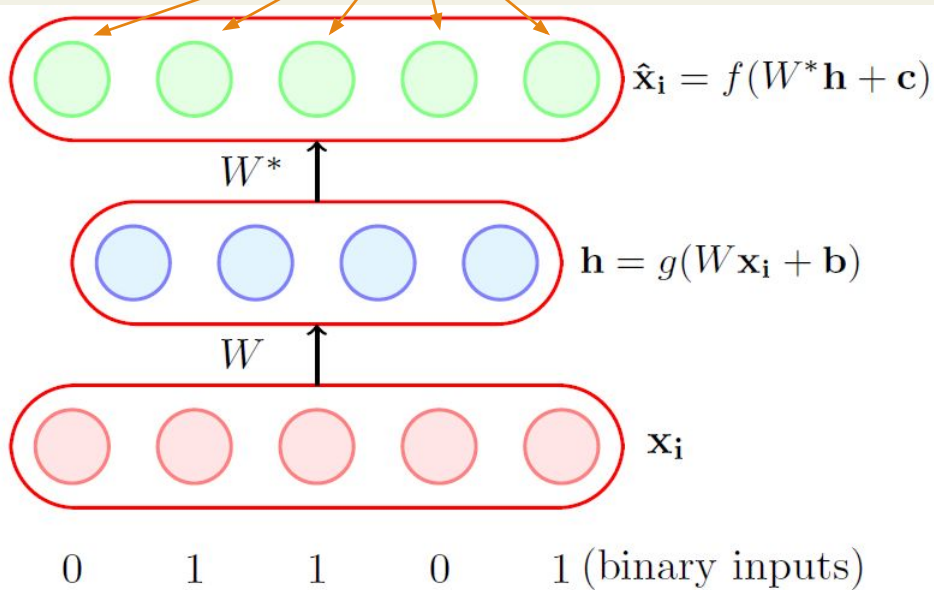


For binary inputs

- In the output layer, we have logistic units, because we have to reconstruct binary values.
- Sample input: [0 1 1 0 1]
- Sample output: [0.2 0.75 0.9 0.11 0.8]
- **Can we use the squared error like in previous case?**
- **Yes.**
- **But cross-entropy loss proves better.**

The gradients

Logistic
units



For binary inputs

- Cross-entropy loss for 1 example is given by:
$$-(y \cdot \log \hat{y} + (1 - y) \cdot \log(1 - \hat{y}))$$

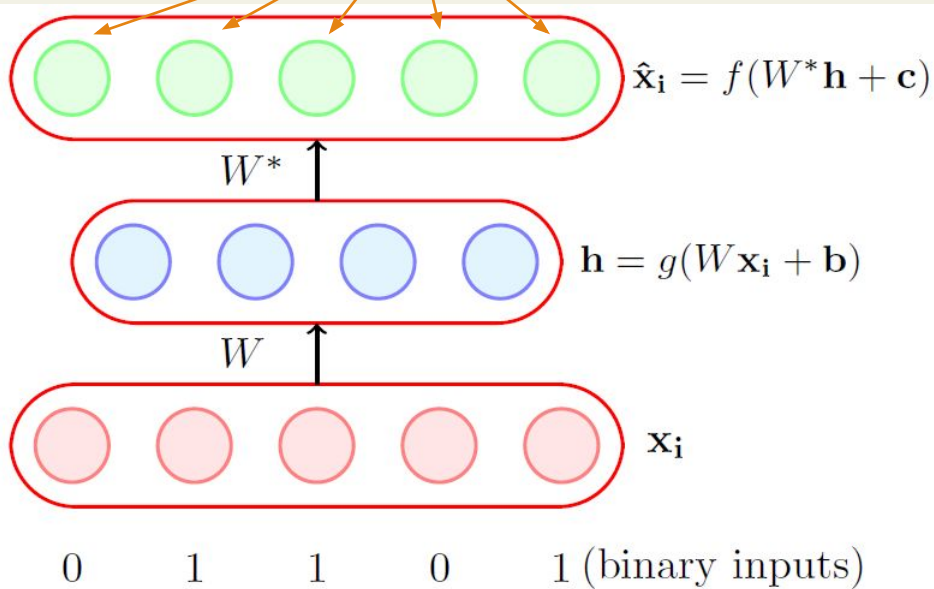
Here y : *Ground truth label*,
 $\hat{y}$: *Predicted probability*

- So, our loss will become:

$$\min\left\{-\sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))\right\}$$

The gradients

Logistic
units



For binary inputs

- So, our loss will become:

$$\min \left\{ - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log (1 - \hat{x}_{ij})) \right\}$$

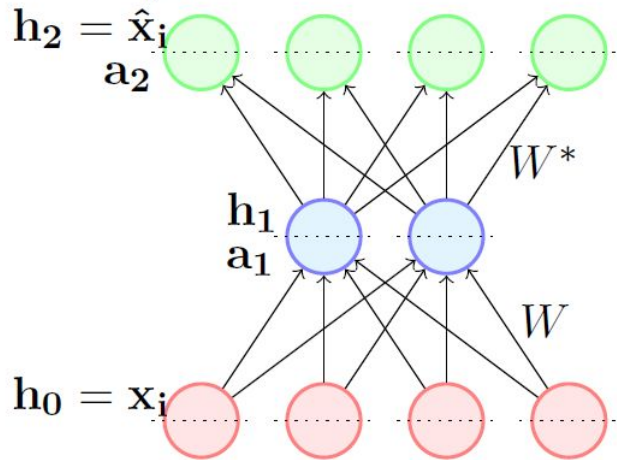
- Loss will be minimized when $x_{ij} = \hat{x}_{ij}$

- For backpropagation we again need to find:

$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} \text{ and } \frac{\partial \mathcal{L}(\theta)}{\partial W}$$

The gradients

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



- $$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial W^*}$$
- $$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial W}$$

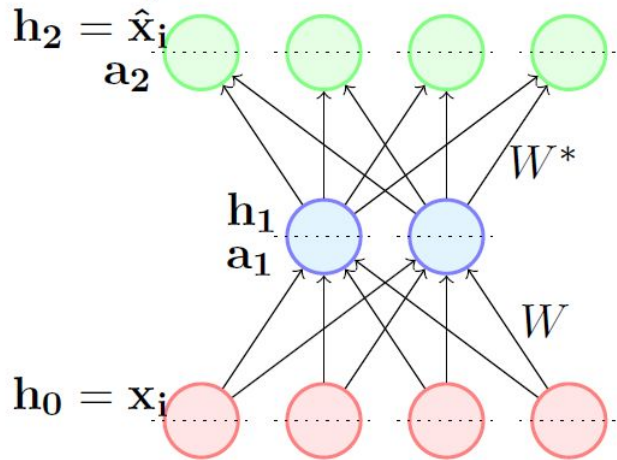
Same as in normal backpropagation derivation.

- We have to calculate:

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_2} \text{ and } \frac{\partial h_2}{\partial a_2}$$

The gradients

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



- We have to calculate:

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_2} \text{ and } \frac{\partial h_2}{\partial a_2}$$

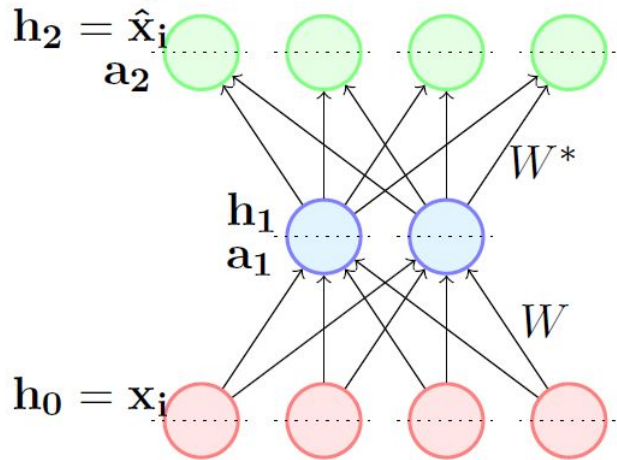
- Here $h_2 = \hat{x}_i$ is a vector.

$$h_2 = \begin{bmatrix} h_{21} \\ h_{22} \\ \dots \\ h_{2j} \\ \dots \\ h_{2n} \end{bmatrix}$$

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_2} = \begin{pmatrix} \frac{\partial \mathcal{L}(\theta)}{\partial h_{21}} \\ \frac{\partial \mathcal{L}(\theta)}{\partial h_{22}} \\ \vdots \\ \frac{\partial \mathcal{L}(\theta)}{\partial h_{2n}} \end{pmatrix}$$

The gradients

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



- We have to calculate:

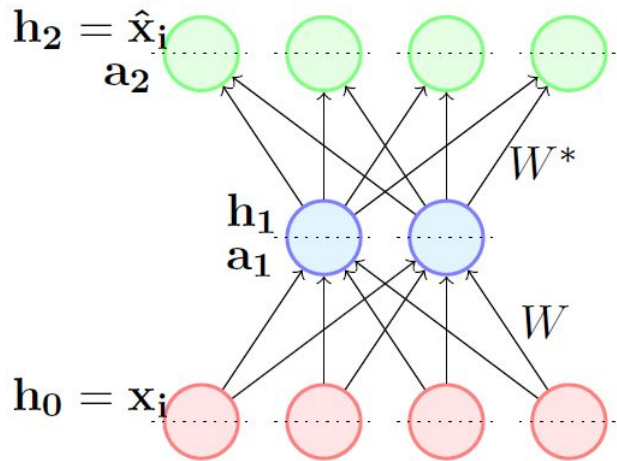
$$\frac{\partial \mathcal{L}(\theta)}{\partial h_2} \text{ and } \frac{\partial h_2}{\partial a_2}$$

- $\hat{x}_{ij} = h_{2j}$
- Hence $\frac{\partial \mathcal{L}(\theta)}{\partial h_{2j}} = \frac{\partial \mathcal{L}(\theta)}{\partial \hat{x}_{ij}}$

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_{2j}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}$$

The gradients

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



- We have to calculate:

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_2} \text{ and } \frac{\partial h_2}{\partial a_2}$$

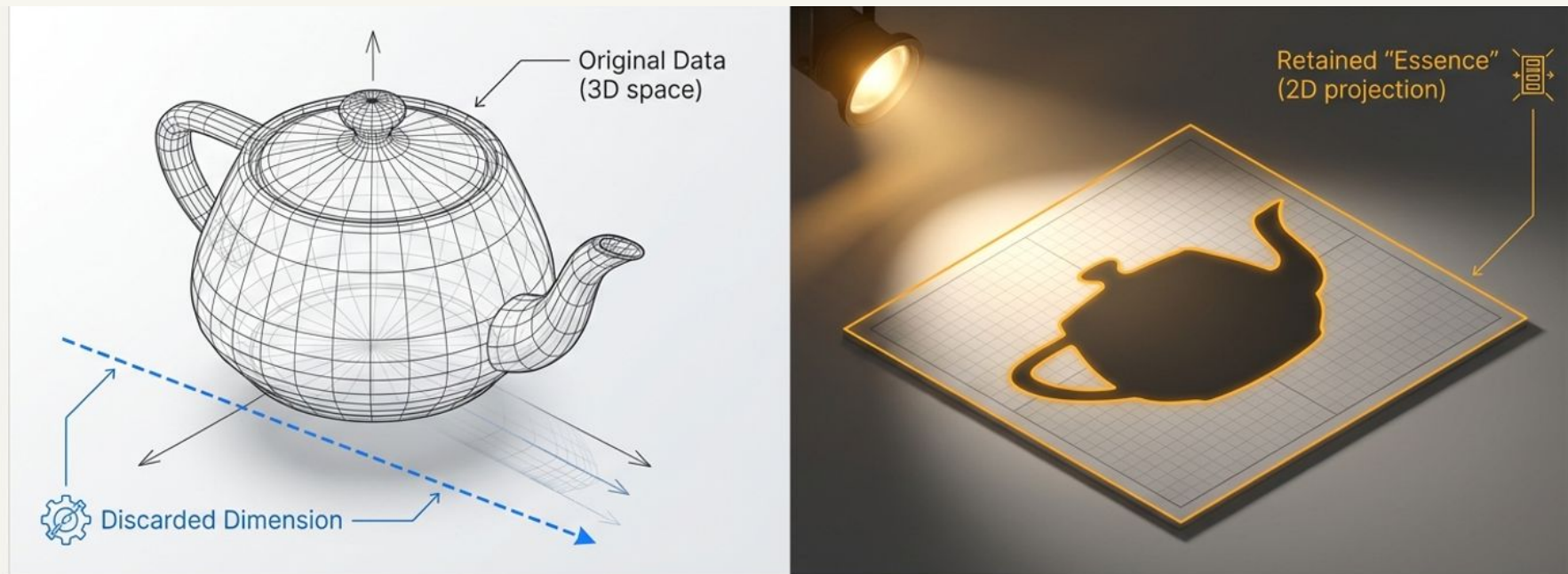
- $\hat{x}_{ij} = h_{2j} = \sigma(a_{2j})$

- Hence $\frac{\partial h_{2j}}{\partial a_{2j}} = \frac{\partial [\sigma(a_{2j})]}{\partial a_{2j}}$

$$\frac{\partial h_{2j}}{\partial a_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$$

Connecting Back to PCA

The connection between PCA and Autoencoders



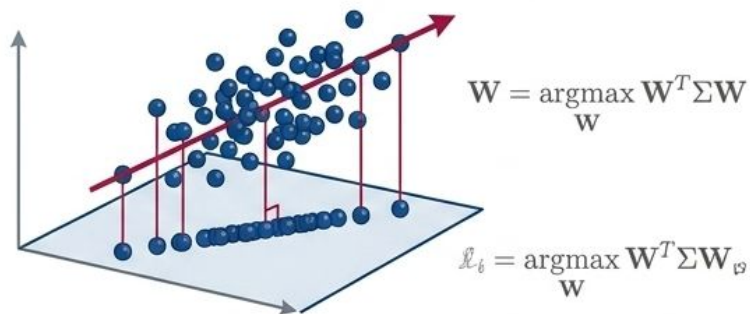
Both tools are designed for **Dimensionality Reduction**.

Imagine taking a 3D object and trying to find the perfect 2D shadow that still looks like the original object. You intentionally lose one dimension to compress the data, but you must keep the “essence.”

The shared goal: Discard dimensions while preserving the recognizable shape of the data.

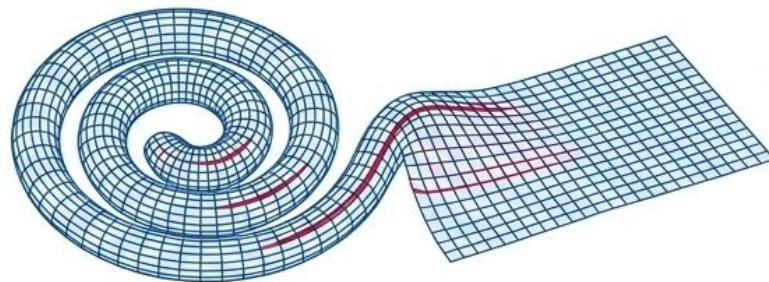
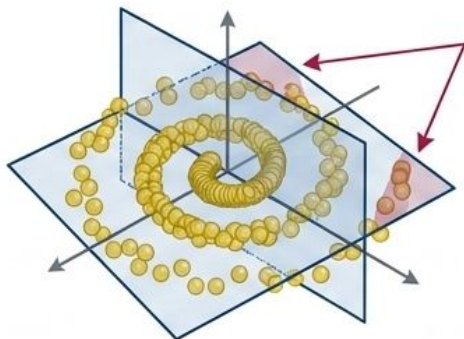
The Baseline: PCA & The Linear Constraint

A



PCA finds straight lines that capture the most "stretch" in data. It maximizes variance but is mathematically constrained to linear transformations.

B



The Limitation: Real-world data is non-linear. Flat planes cannot capture curved manifolds. Dimensionality reduction requires a non-linear solution.

**If we restrict an
Autoencoder to only use
linear activations and
squared error loss,
what do we get?**

Linear Autoencoders vs. Deep Autoencoders

The Equivalence Rule: When constrained to specific rules, an Autoencoder's encoder weight matrix (W) spans the exact same geometric space as PCA's eigenvectors.

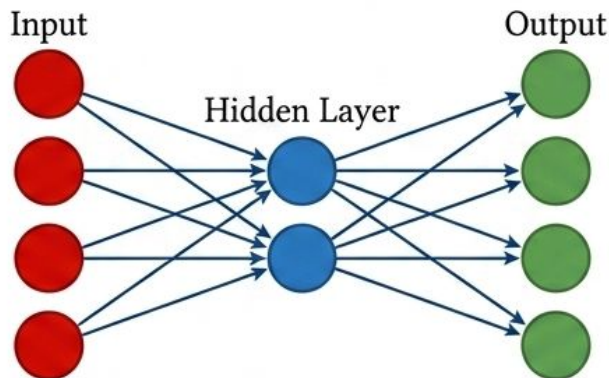
Attribute	Linear Autoencoder (PCA)	Deep Autoencoder
Activation Rule	Linear Activations Only (No curved math)	Non-linear (ReLU, Sigmoid) allowing curved manifolds
Penalty Rule	Squared Error Loss Function (Minimizing $(x - \hat{x})^2$)	Cross-Entropy or Custom Loss
Data Rule	Normalized Inputs (Zero mean)	Normalized Inputs

Conclusion: A Linear Autoencoder is just PCA. A Deep Autoencoder is PCA with superpo 

Regularizing Autoencoders

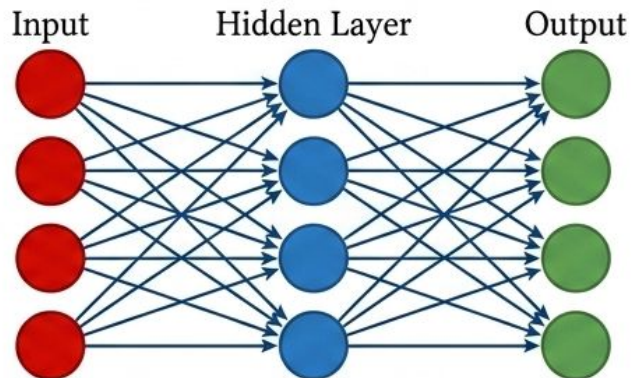
The Capacity Problem: Undercomplete vs. Overcomplete

Undercomplete Architecture



- Condition: $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$
- Mechanism: The bottleneck acts as a physical constraint.
- Result: Forces the network to discard noise and perfectly compress data to learn the underlying structure. This is the ideal scenario.

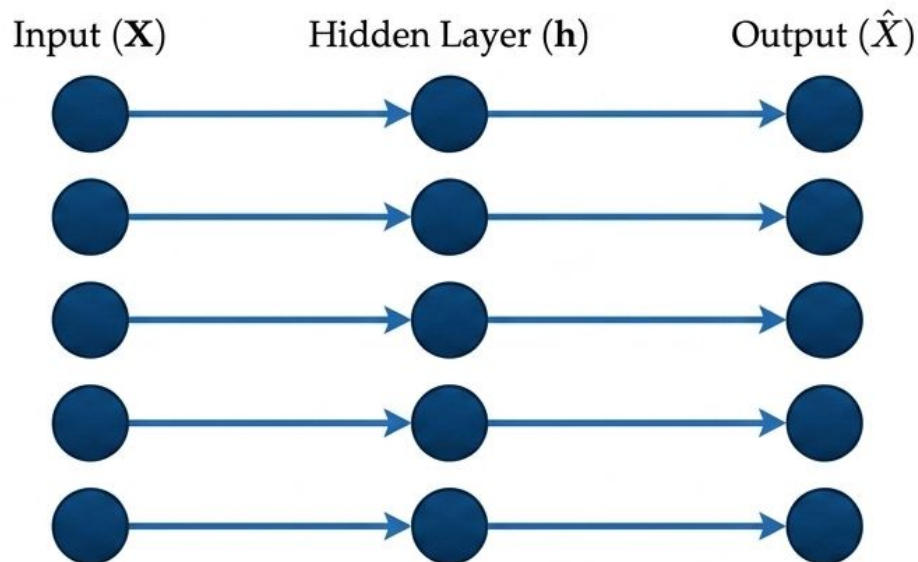
Overcomplete Architecture



- Condition: $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- Mechanism: The bottleneck is wider than the data itself.
- Result: Excess capacity introduces a structural vulnerability, paving the way for a mathematically useless encoding.

The Fatal Flaw: The Identity Trap

Trivial Identity Mapping



The Mechanism:

When given more hidden units than input features, the neural network bypasses learning entirely. It learns a trivial encoding by simply copying x_i directly to $\mathbf{h}$, and $\mathbf{h}$ directly to $\hat{x}_i$.

The Consequence:

The network memorizes the data without understanding it. It extracts no underlying features. The reconstruction is mathematically perfect (zero loss), but functionally **useless** as a feature extractor.

Regularization



The Sieve

To prevent the Identity Trap, we must introduce Regularization.



The Mold

Regularization is an intentional obstacle course. By mathematically making the network's life harder, we strip away its ability to memorize, forcing it to discover the true, underlying rules of the data.

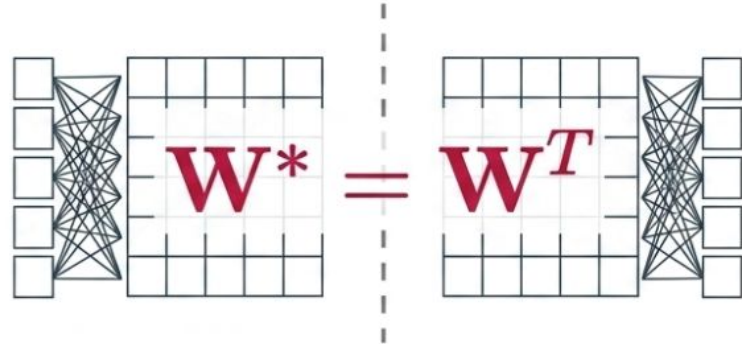
L2 decay vs Tied weights

Constraint by Structure: L2 Weight Decay

$$J(\theta, \mathbf{X}) = \mathcal{L}(\mathbf{X}, \hat{\mathbf{X}}) + \lambda \|\theta\|^2$$

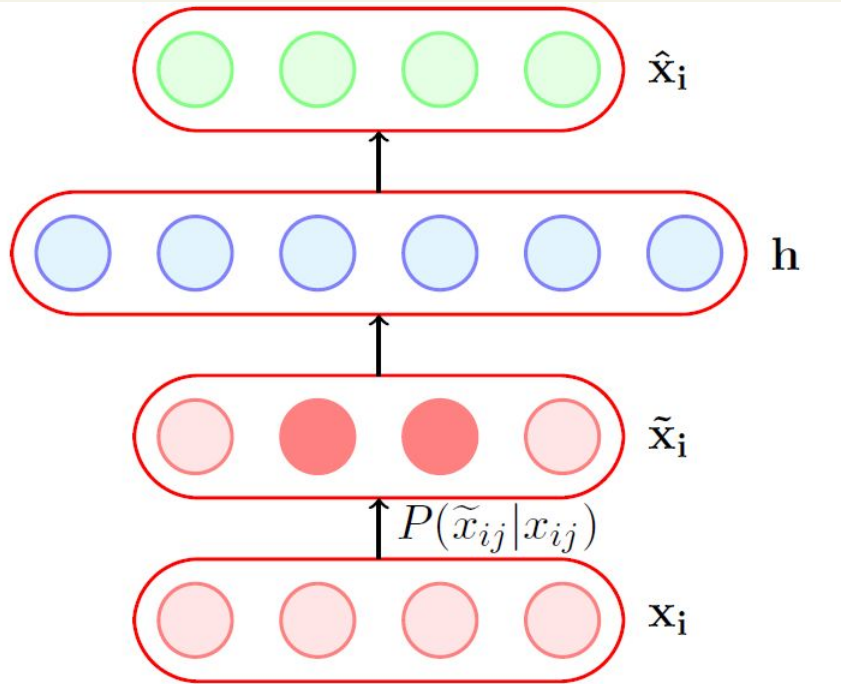
Penalizes large weights. Forces the network to distribute learning across many features rather than relying heavily on a few.

Constraint by Structure: Tied Weights



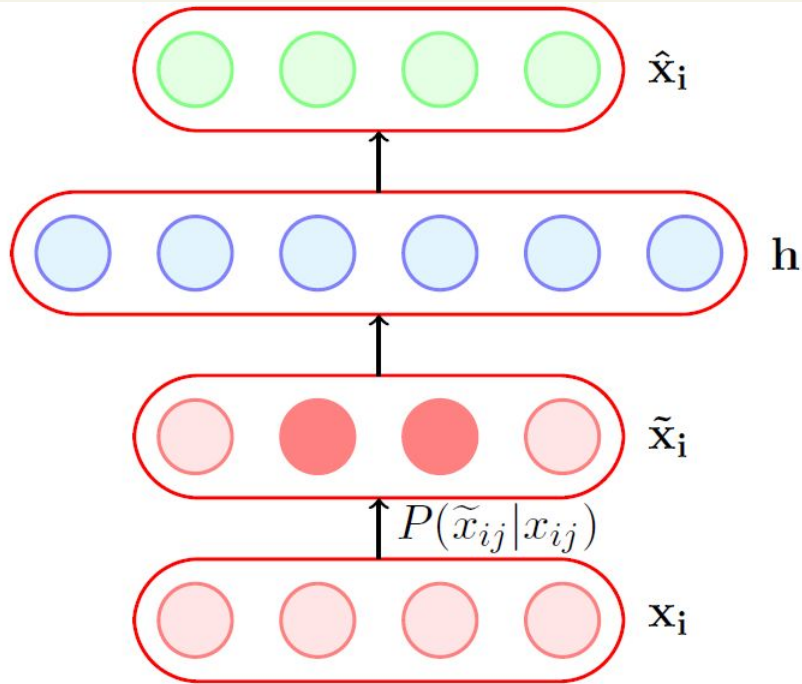
Slashes the model's capacity in half. The decoder is mathematically forced to use the **exact inverse** of the encoder's mapping.

Denoising Autoencoder



- A denoising encoder intentionally corrupts the input.
- Input corruptions follows a probabilistic process $P(\tilde{x}_{ij} | x_{ij})$.

Denoising Autoencoder

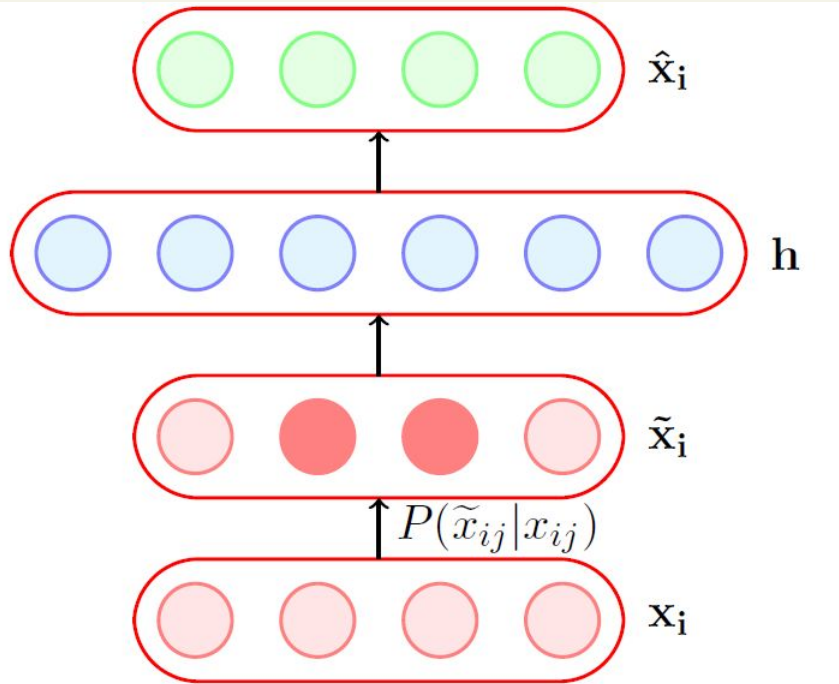


Corruption process can follow:

- $P(\tilde{x}_{ij} = 0 | x_{ij}) = q$
- $P(\tilde{x}_{ij} = x_{ij} | x_{ij}) = 1 - q$
- i.e. with probability q the input is flipped to 0, otherwise retained.

OR

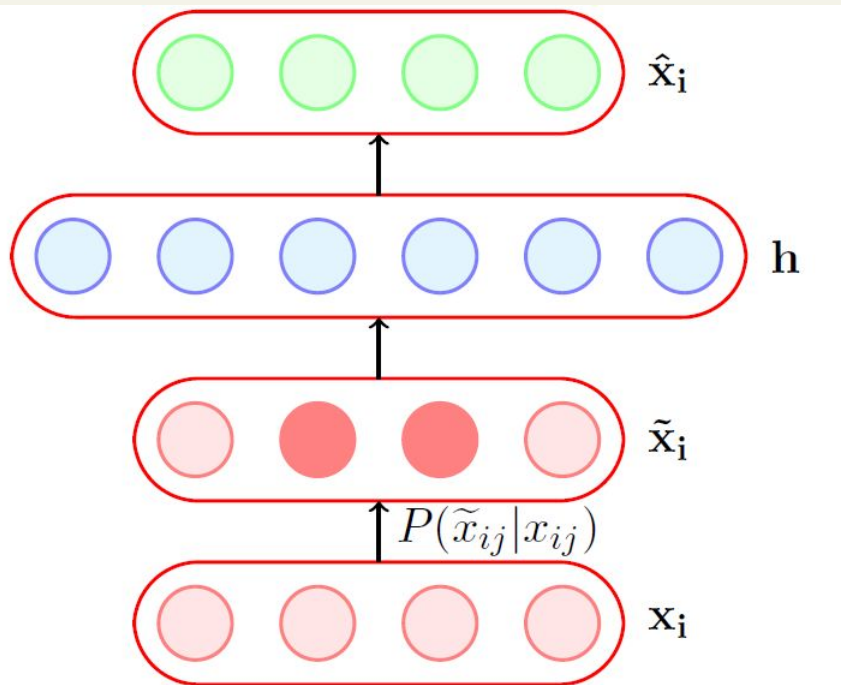
Denoising Autoencoder



Corruption process can follow:

- Gaussian noise - $\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0,1)$
- Will such a corruption prevent the identity mapping?

Denoising Autoencoder



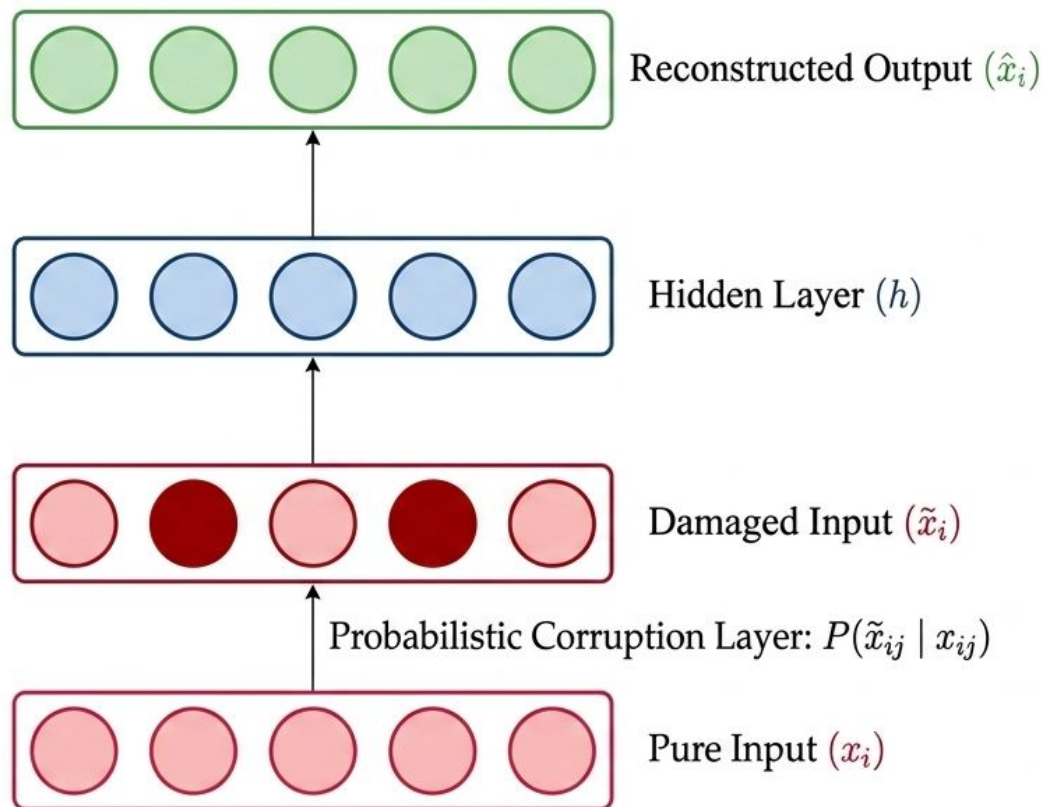
Yes.

- Because **loss** is still being calculated on the basis of x_i **vs** $\hat{x}_i$.
- Simply copying $\tilde{x}_i$ will not reduce the loss.
- Model is now being forced to capture the characteristics of the data.

Some empirical observations

- Denoising AE can learn more meaningful features than vanilla AE.
- Due to noise injection, the neurons are forced to learn the relationships between adjacent values.
- Denoising AE can learn better features than regularized AE.

Denoising Autoencoders: The Corruption Principle



Core Concept:

A denoising encoder intentionally damages the input before processing.

The Loophole Closer:

The loss function calculates the error against the uncorrupted original input (x_i), NOT the damaged input ($\tilde{x}_i$).

The Result:

Simply copying the damaged input ($\tilde{x}_i$) will fail to reduce the loss. The network is physically unable to use the **Identity Trap**. It must learn the underlying rules of the data to fix the damage.

Mechanisms of Denoising

Corruption Profiles:

Binary Flipping

$$P(\tilde{x}_{ij} = 0 \mid x_{ij}) = q$$

With probability q , the input is flipped to 0; otherwise retained.

Gaussian Noise

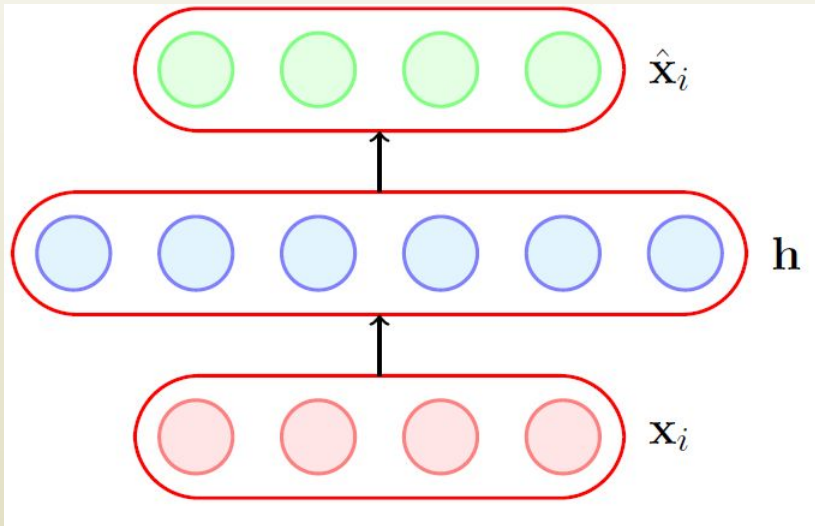
$$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, 1)$$

Continuous noise is added to real-valued inputs.

Empirical Observations

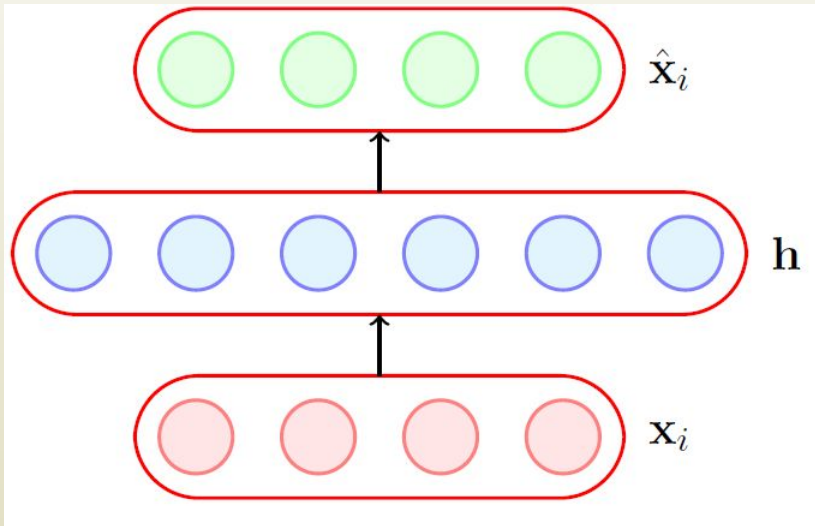
- Because the model cannot copy the input, denoising forces neurons to learn the relationships between adjacent values to 'guess' the missing pieces.
- Denoising Autoencoders consistently learn more meaningful and robust features than Vanilla or Regularized Autoencoders.

Sparse autoencoders



- **Sparsity** ensures most of the neurons will not fire.
- A hidden sigmoid neuron will have values between 0 & 1.
- Output near 1 → Neuron fired
- Output near 0 → Neuron didn't fire.

Sparse autoencoders

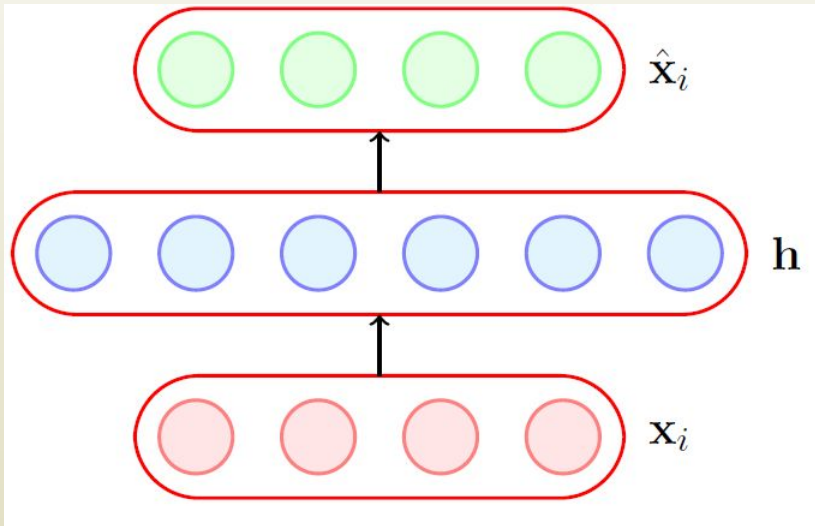


- The average value of the activation of a neuron l is given by

$$\hat{\rho}_l = \frac{1}{m} \sum_{i=1}^m h(x_i)_l$$

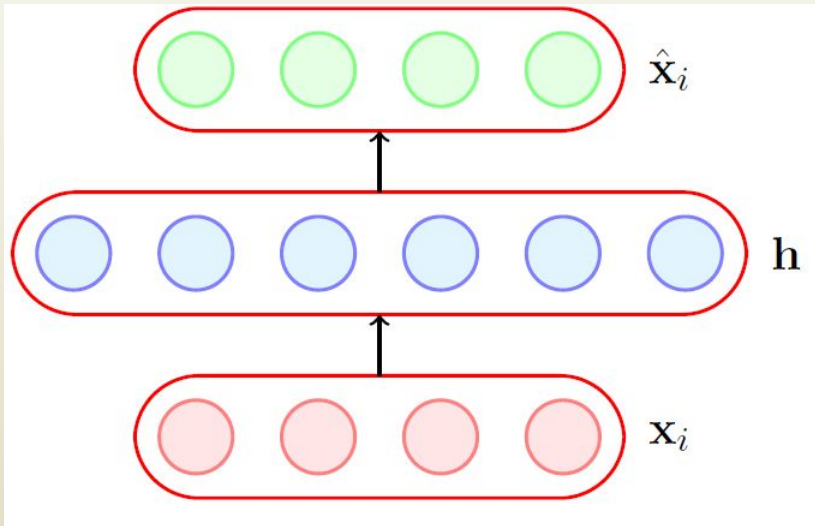
- If the neuron l is sparse (i.e. mostly inactive) then

Sparse autoencoders



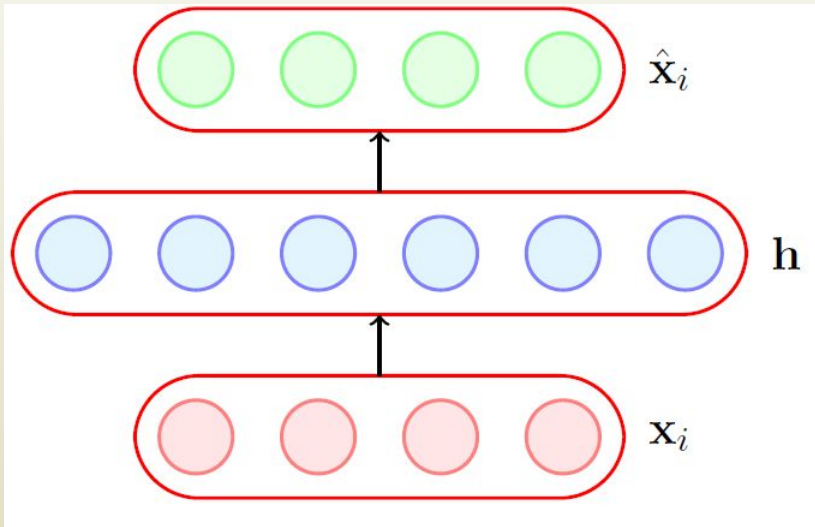
- **Remember:**
- Overfitting happens because the parameters have a lot of freedom.
- Regularization constrains this freedom.

Sparse autoencoders



- **Remember:**
- Overfitting happens because the parameters have a lot of freedom.
- Regularization constrains this freedom.
- Sparse autoencoder has a sparsity parameter ρ .
- Typically, ρ is very small (like 0.005).
- Sparse AE forces average activation $\hat{\rho}_l = \rho$.

Sparse autoencoders



- To ensure sparsity the regularization term $\Omega(\theta)$ is added to squared error loss or cross-entropy loss.
- $\hat{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$

- $$\Omega(\theta) = \sum_{l=1}^k \rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_l}$$

Synthesis: The Autoencoder Architecture Matrix

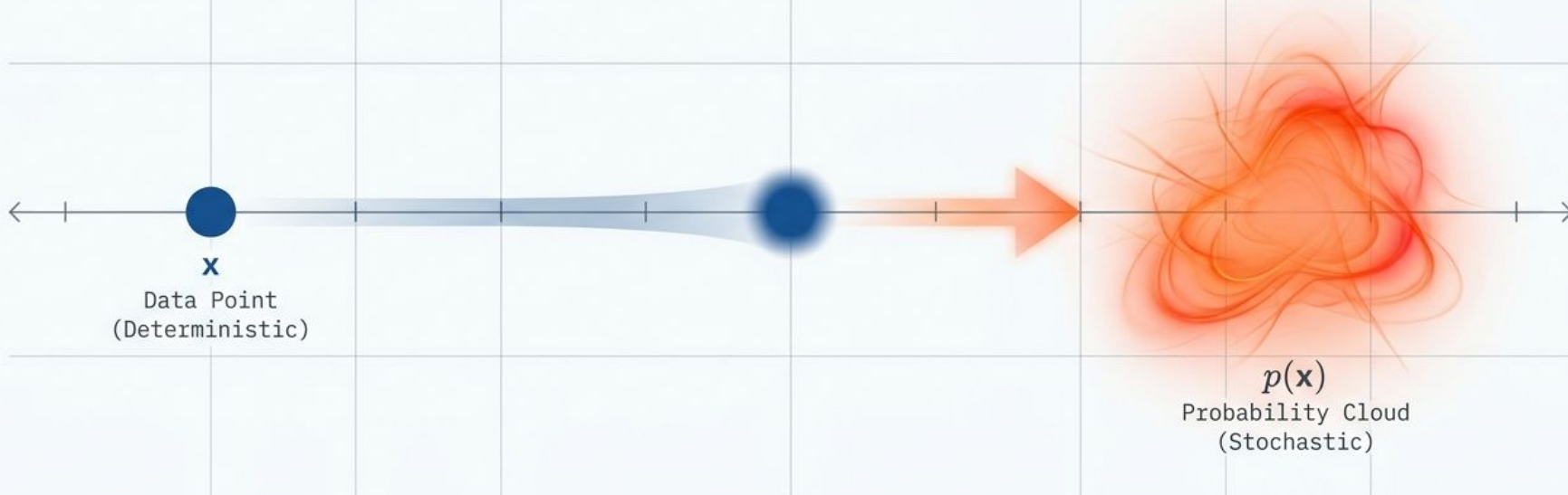
	Undercomplete (Vanilla)	Denoising (DAE)	Sparse (SAE)
Core Mechanism	Dimensional bottleneck	Probabilistic input corruption	Activation suppression
Primary Constraint	$\dim(h) < \dim(x_i)$	$P(\tilde{x}_{ij} \mid x_{ij})$	$\hat{\rho}_l \approx 0$
Method to Avoid Identity Trap	Physical lack of capacity	Loss calculated against pure x_i	Forced neural specialization

Final Takeaway: Autoencoders do not merely compress data—through strategic constraints, they are forced to discover its fundamental laws.

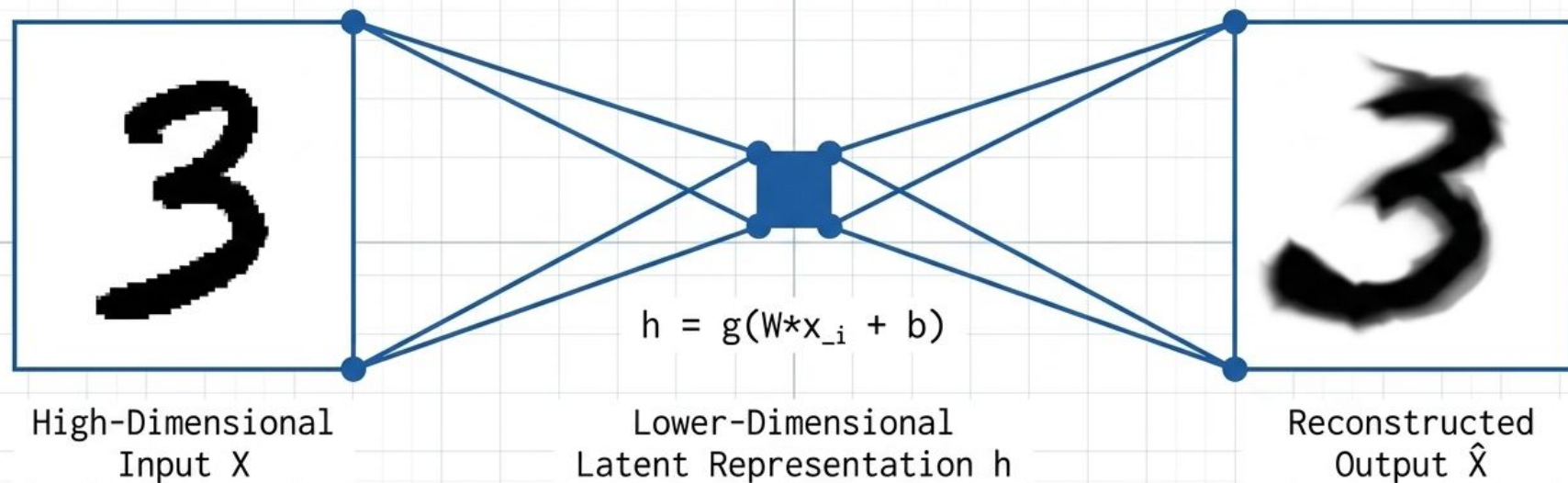
Variational Autoencoders

The Generative Blueprint: Transitioning from Autoencoders to VAEs

Bridging the conceptual gap between deterministic data compression and probabilistic generative modeling.



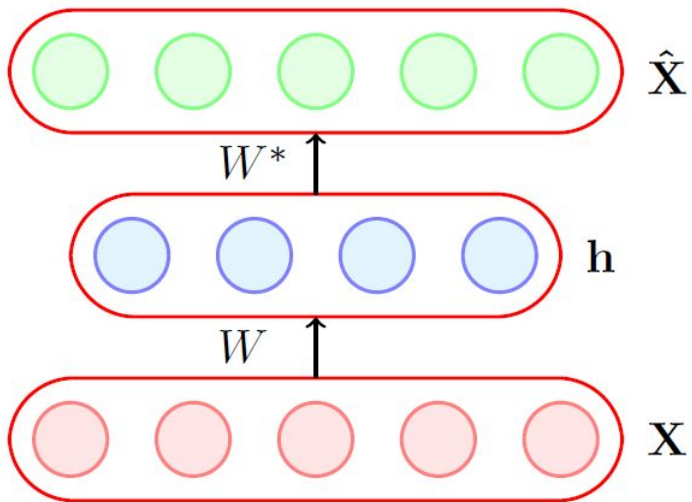
The Baseline: Standard Autoencoders



- A standard AE maps high-dimensional input X (e.g., a 784-dimensional MNIST pixel grid) into a lower-dimensional latent representation h .
- Objective: Minimize reconstruction loss so that $\hat{X}$ perfectly matches X .

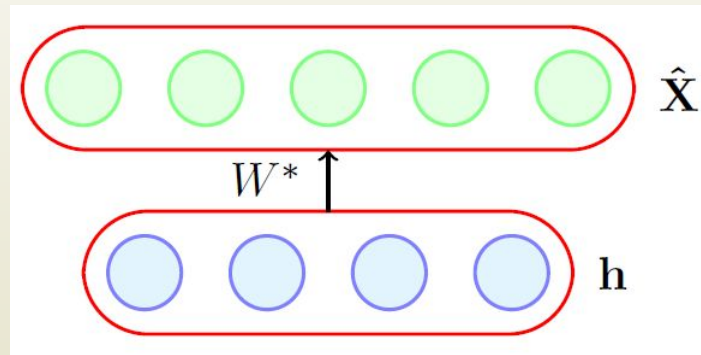
Can we do regeneration with AE?

Can we simply pass a hidden h to **decoder** and reconstruct $\hat{X}$?



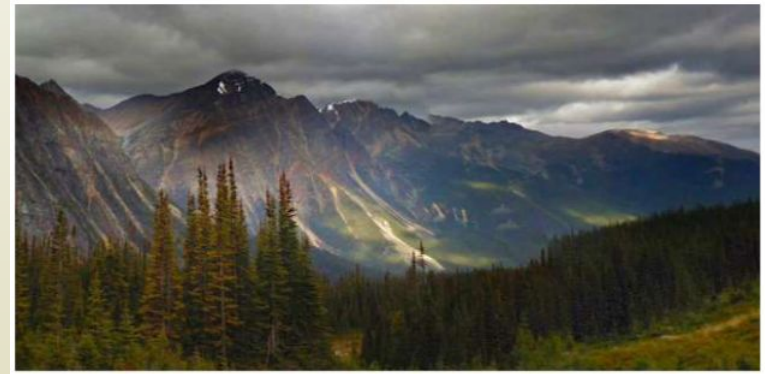
$$\mathbf{h} = g(W\mathbf{X} + \mathbf{b})$$

$$\hat{\mathbf{X}} = f(W^*\mathbf{h} + \mathbf{c})$$



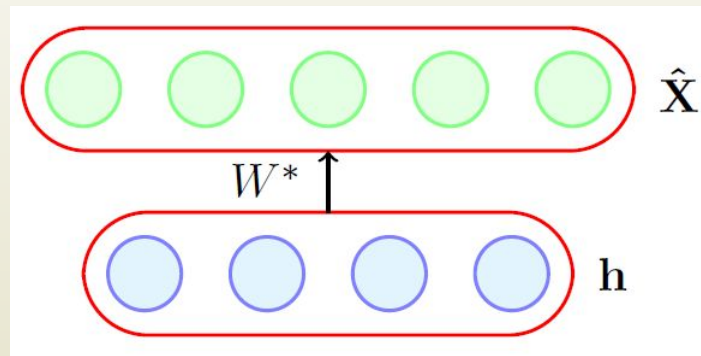
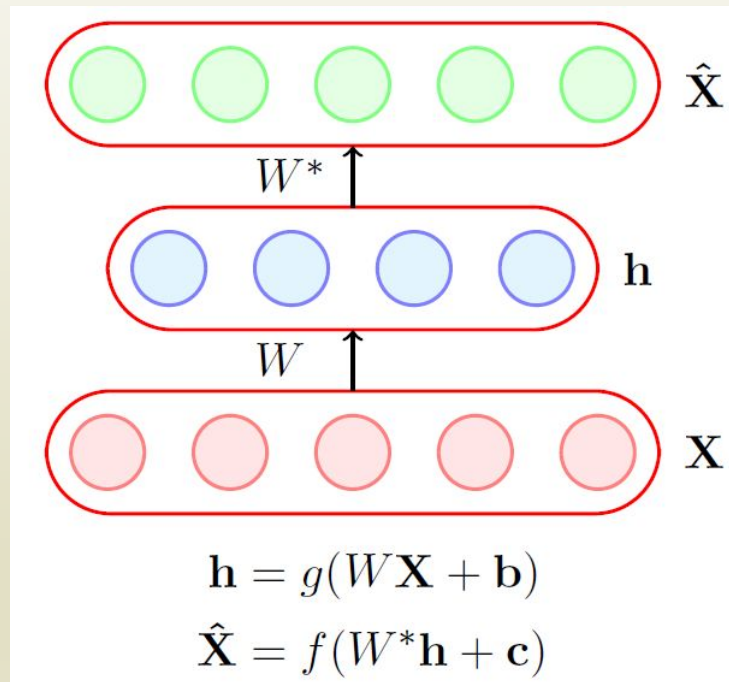
In other words

From a large collection of landscapes, can a network learn to generate a new landscape?



Can we do regeneration with AE?

Can we simply pass a hidden h to **decoder** and reconstruct $\hat{X}$?



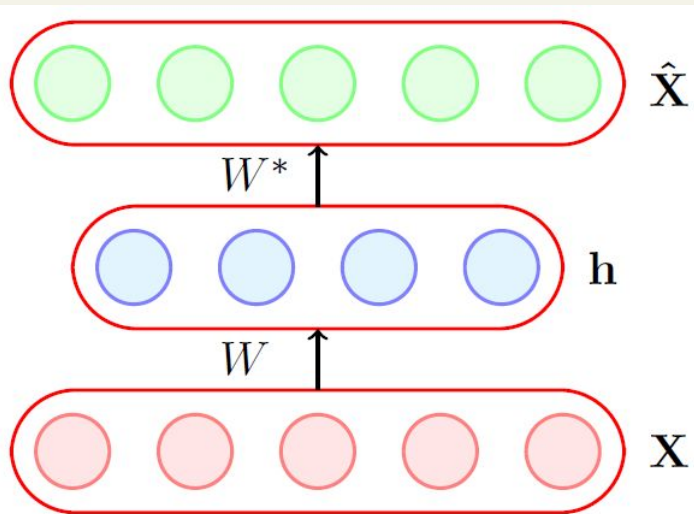
Theoretically, YES.

But practically it doesn't work that way.

WHY?

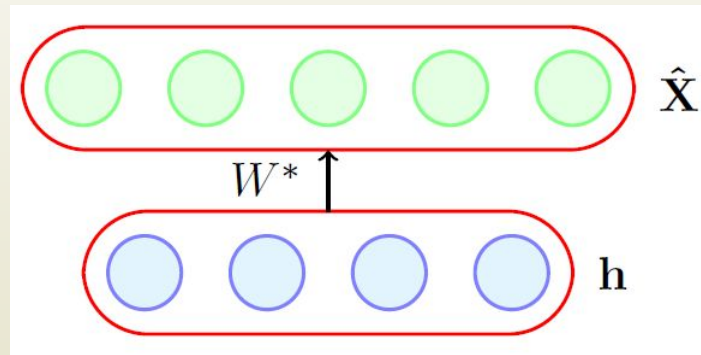
Can we do regeneration with AE?

Can we simply pass a hidden h to **decoder** and reconstruct $\hat{X}$?



$$\mathbf{h} = g(W\mathbf{X} + \mathbf{b})$$

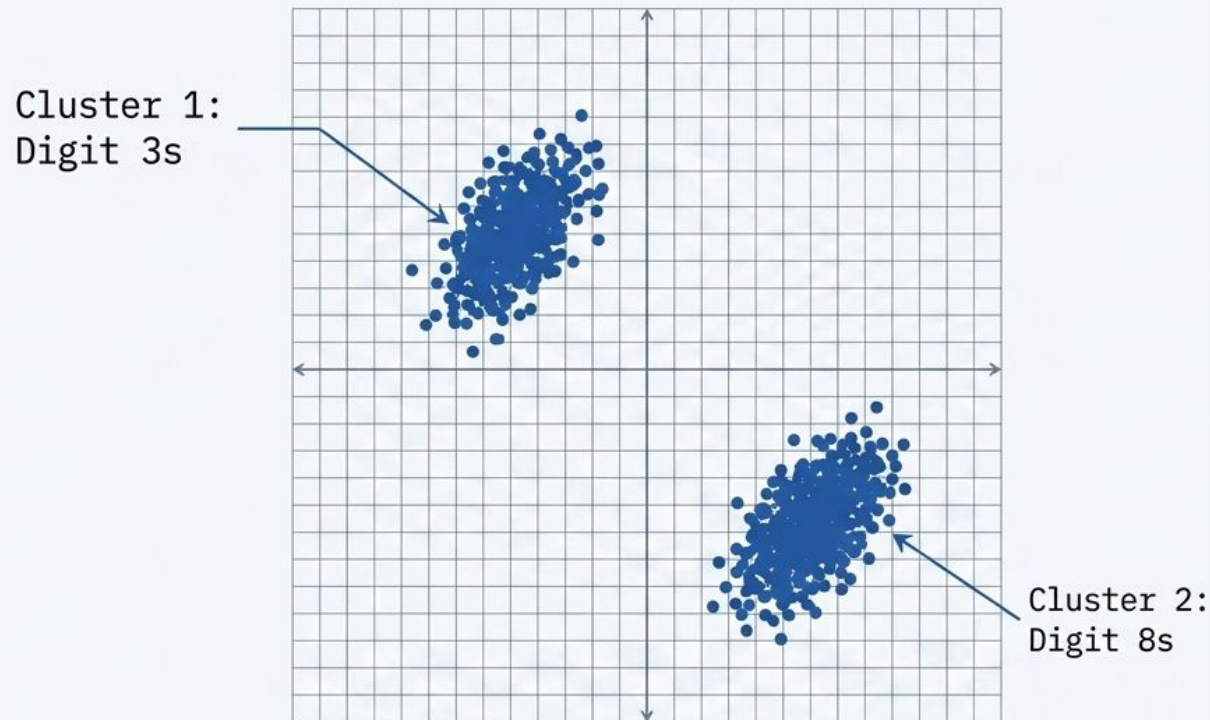
$$\hat{\mathbf{X}} = f(W^*\mathbf{h} + \mathbf{c})$$



- h is very high-dimensional.
- Out of all the possible values of h , only a few vectors correspond to a meaningful representation.



The Power of Deterministic Compression

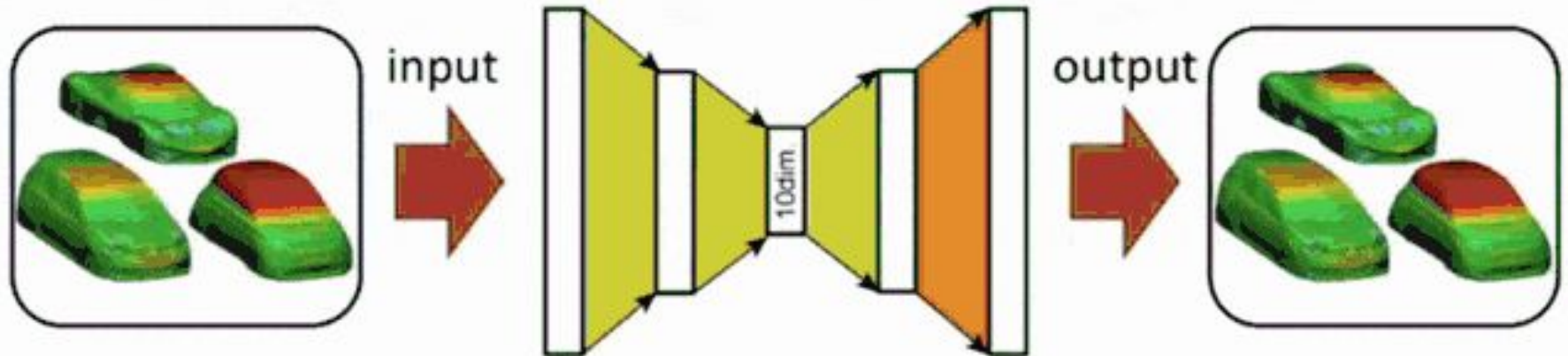


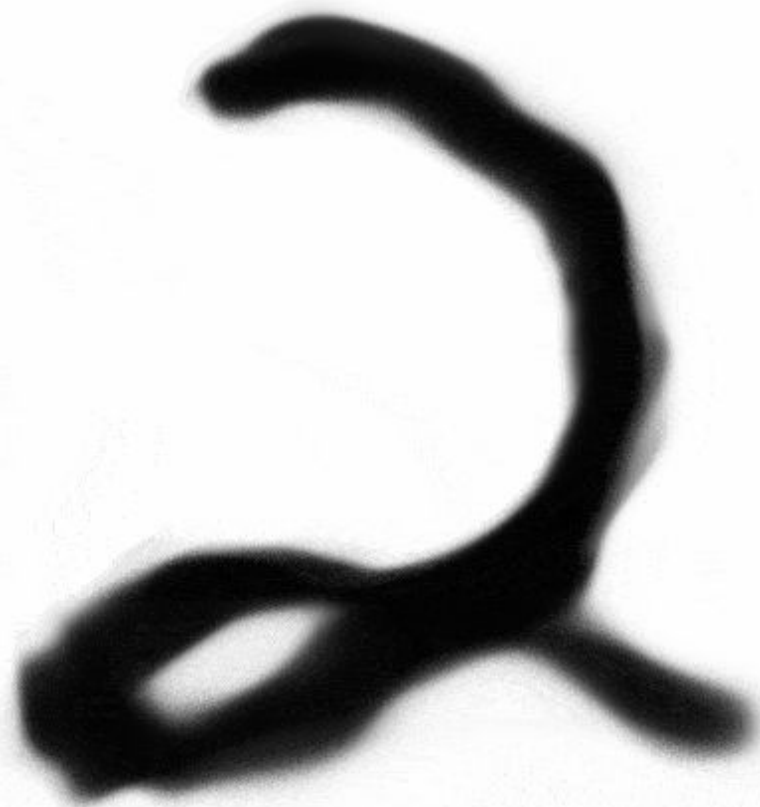
Standard AEs excel at abstraction.

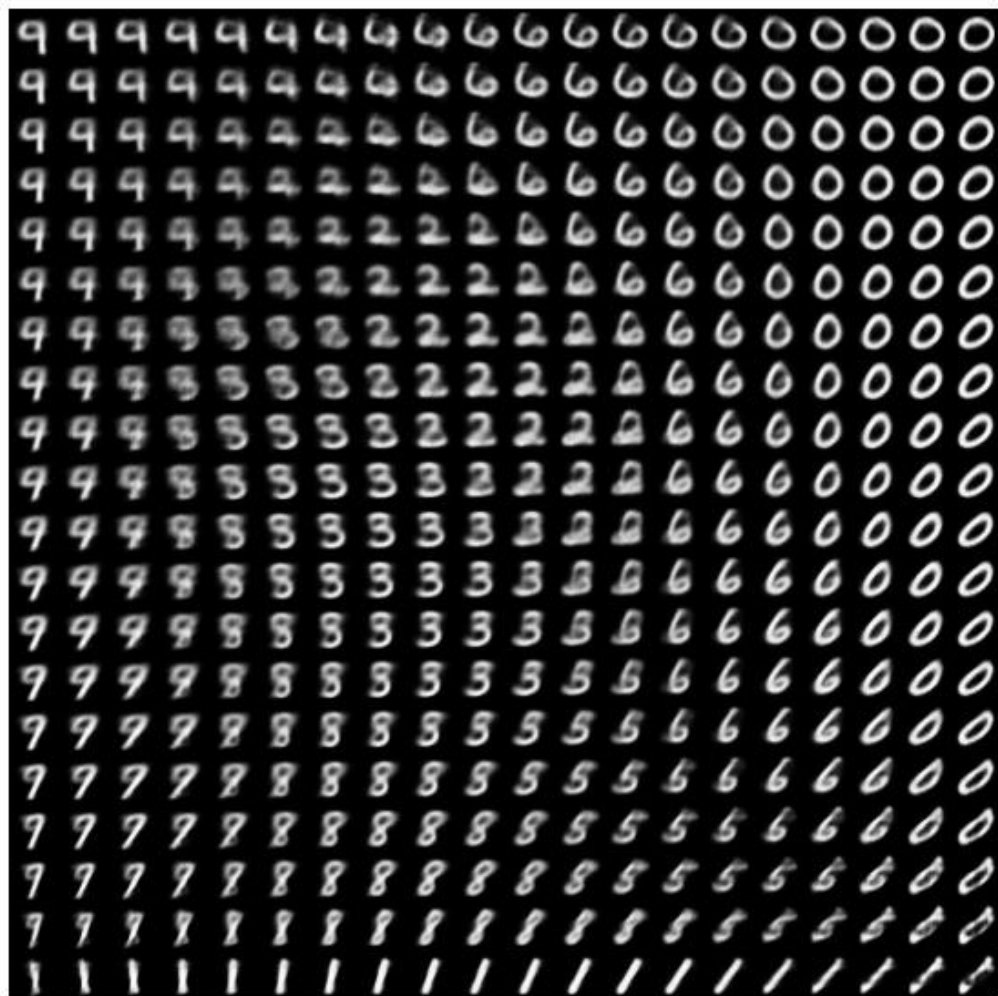
They learn a deterministic, low-loss encoding of the input data.

Every exact input $\mathbf{x}_i$ maps to a single, rigid coordinate in the latent space $\mathbf{h}$.

We then trained an autoencoder network such that the input and the output of the network is as same as possible for the parameterized 3D shapes.

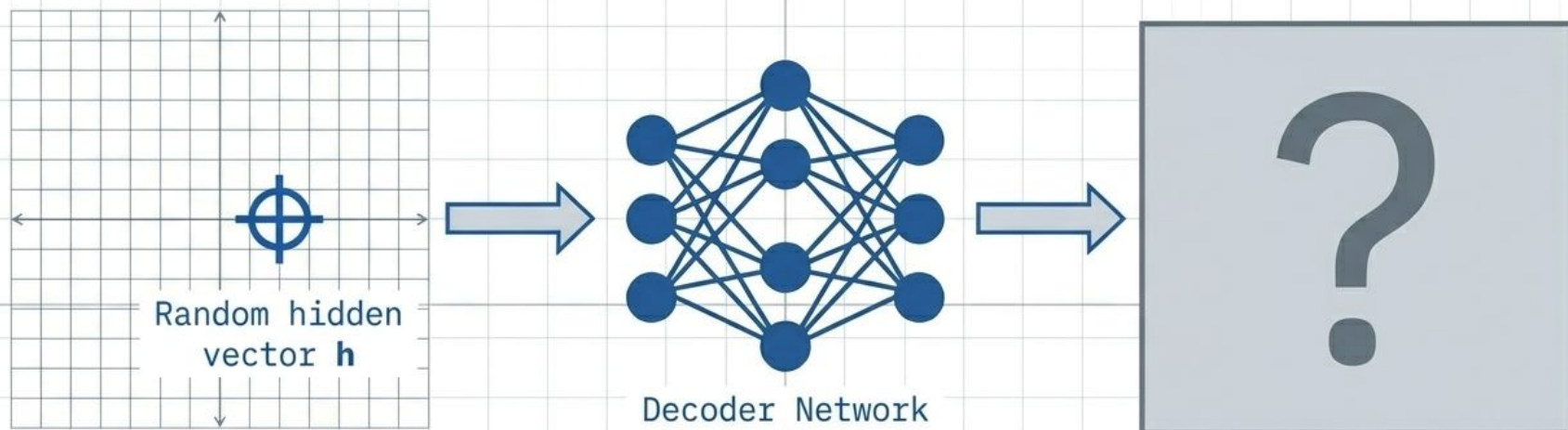








The Generative Goal: The “What If?”



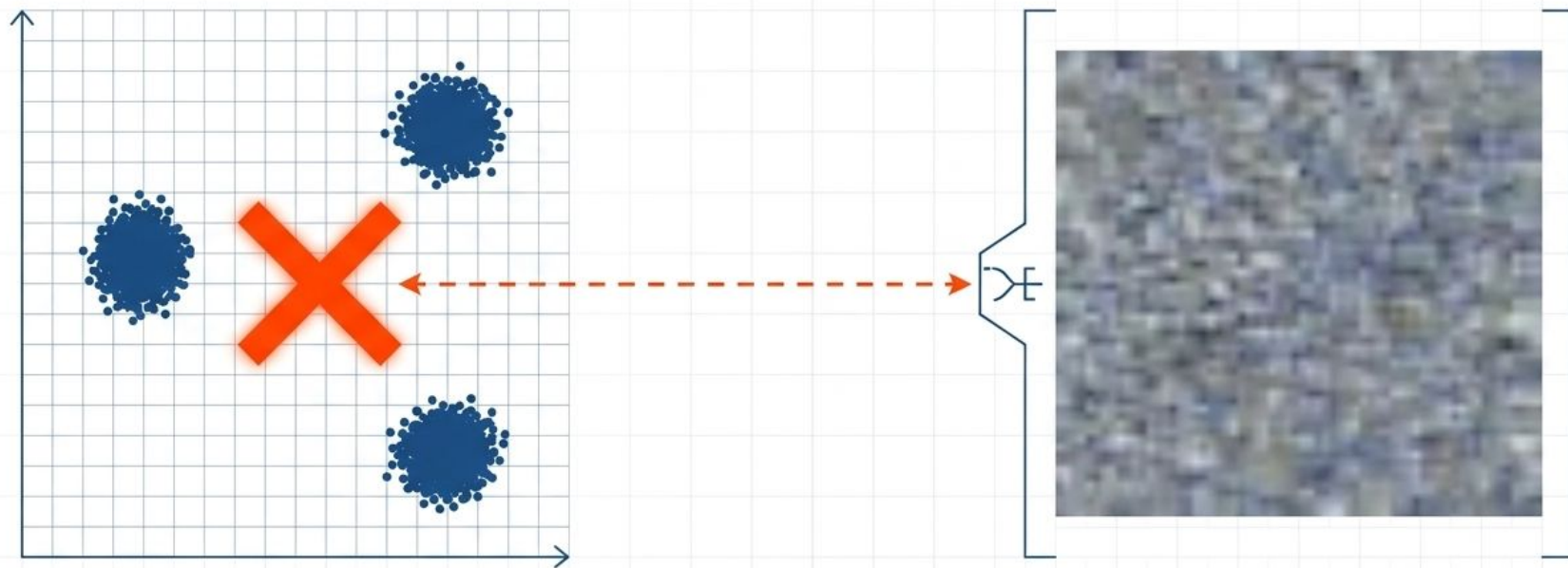
The Question

Can we discard the encoder entirely, pick a random hidden vector h , feed it to the decoder, and generate a brand new, realistic image from thin air?

The Reality

In principle, yes.
In practice, no.

The Flaw: Dead Zones in Latent Space



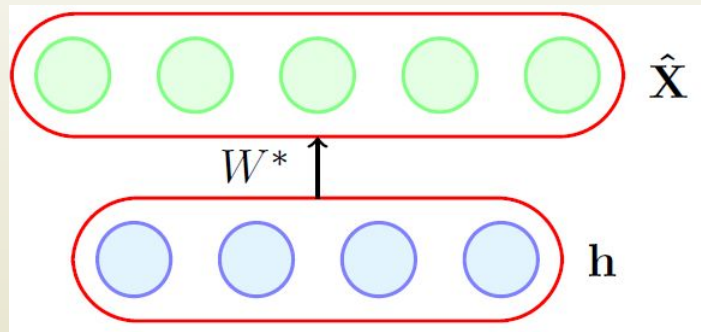
Sparse Mapping

The network only knows how to decode the exact points it memorized during training.

Discontinuous Space

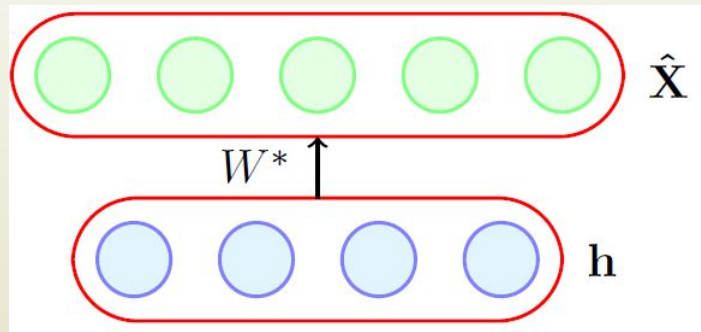
Pick a point in the empty space between clusters, and the decoder fails entirely, outputting meaningless noise.

From points to distributions.



- Only highly likely values of h should be fed to the decoder.
- We need to sample h from $P(h | X)$.

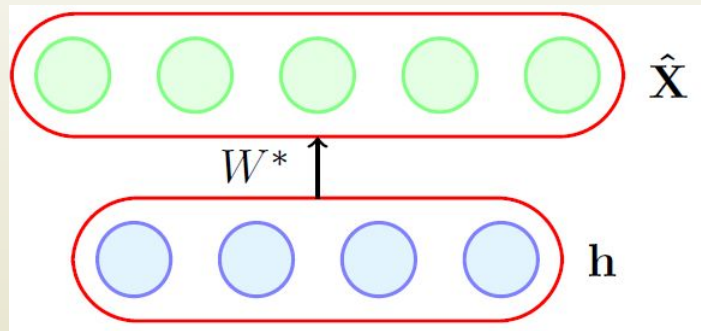
From points to distributions.



- Only highly likely values of h should be fed to the decoder.
- We need to sample h from $P(h | X)$.

But AE learns hidden representation h and not $P(h|X)$.

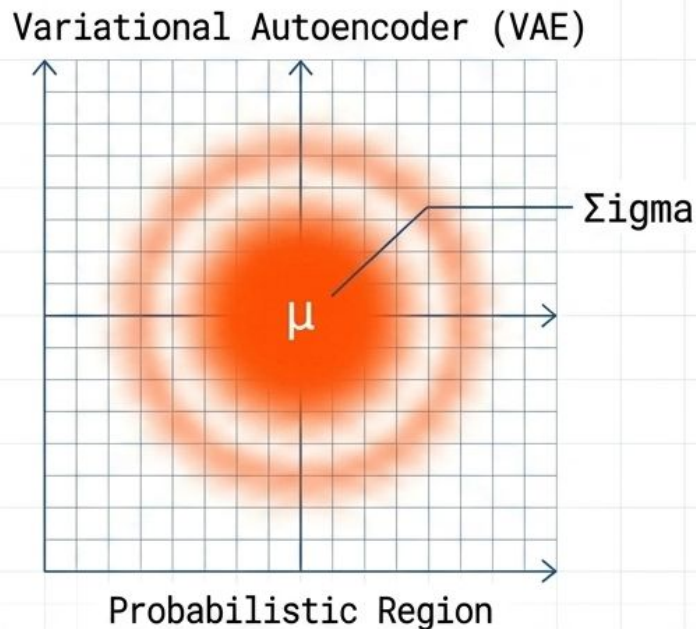
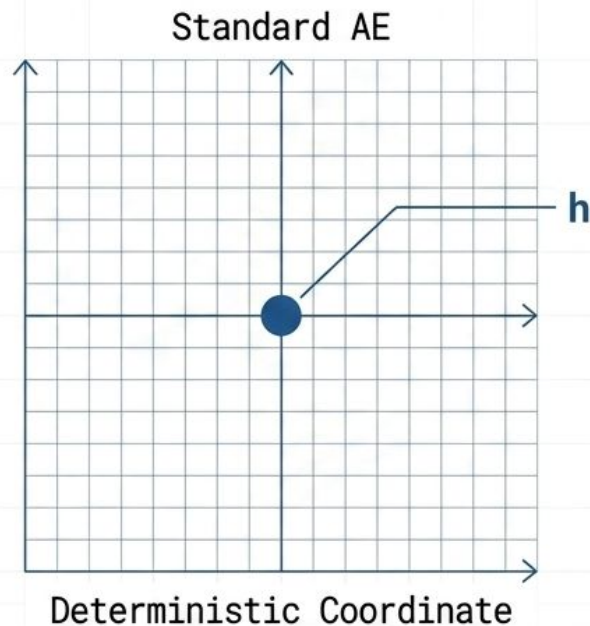
From points to distributions.



- Only highly likely values of h should be fed to the decoder.
- We need to sample h from $P(h | X)$.

Decoder is also
deterministic, i.e. it learns
X from h and not $P(X|h)$.

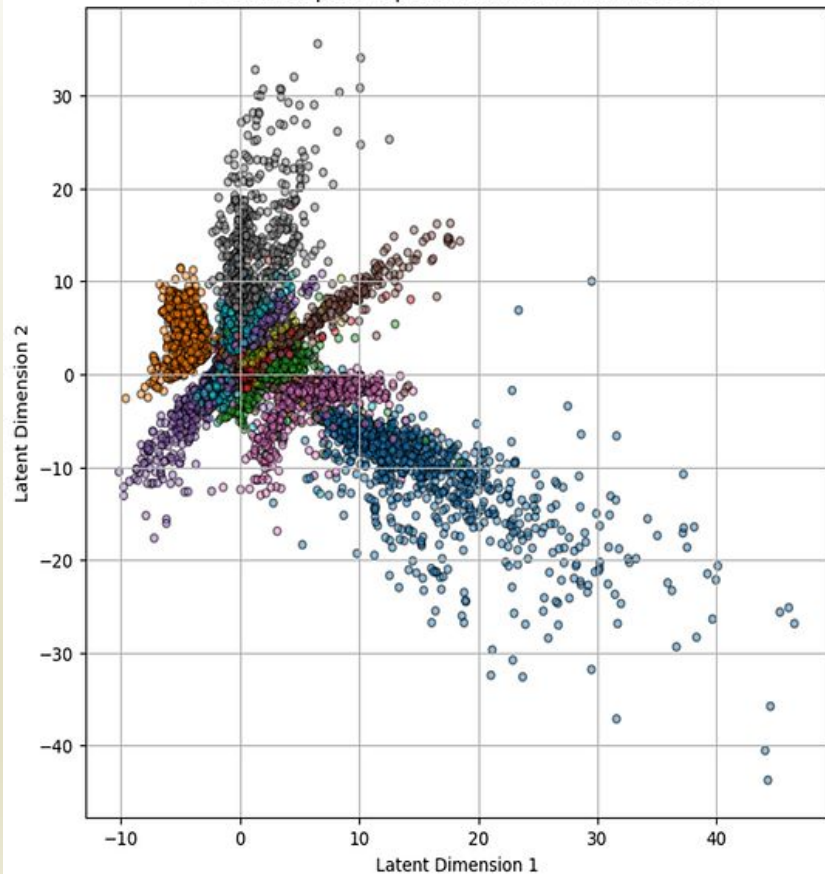
The Intuition Shift: From Points to Distributions



Core Shift: To generate new data, we need a continuous, well-behaved latent space. Instead of mapping an input to a single deterministic point, we map it to a probabilistic region.

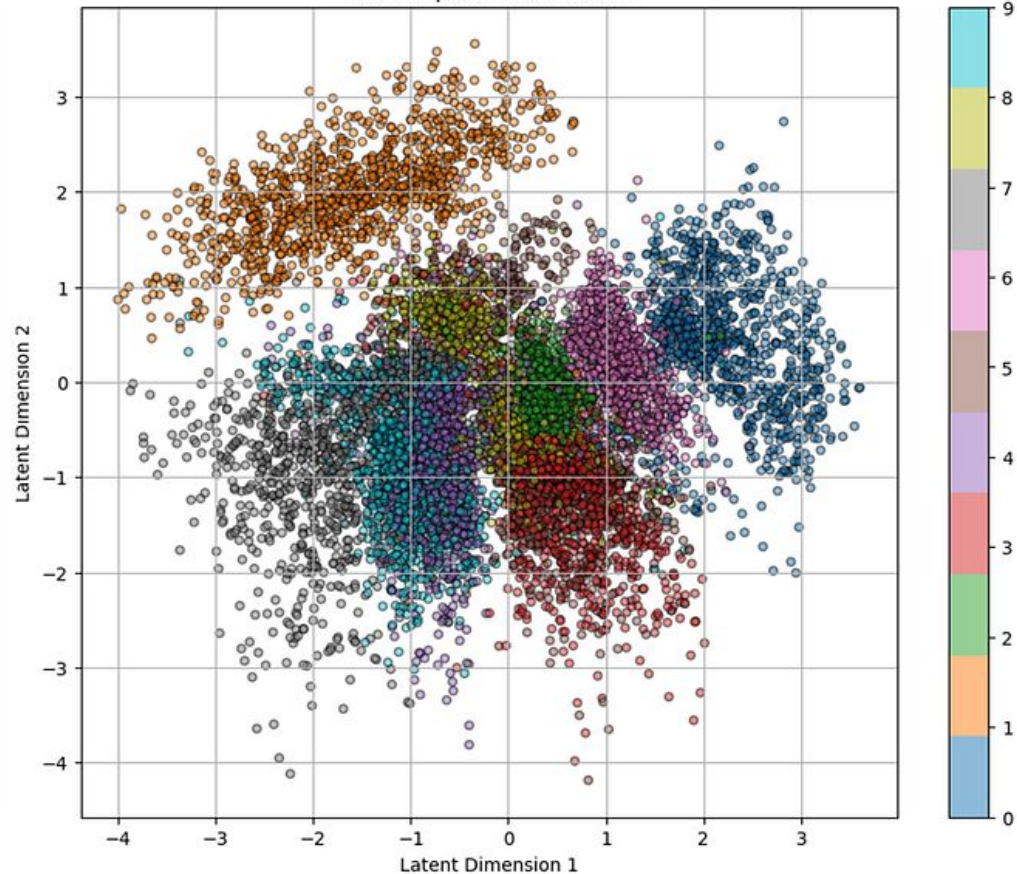
Autoencoder

2D Latent Space Representation of MNIST Test Data



Variational Autoencoder

Latent Space Visualization



From points to distributions



Figure: Abstraction

- Given an X , find a hidden representation z (or h).
- Given a hidden representation z generate **an** X .
- Probabilistically, learn $P(z|X)$ and $P(X|z)$.

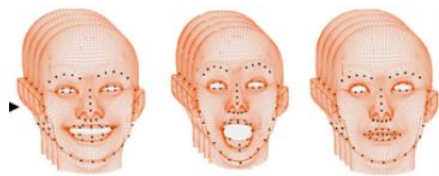


Figure: Generation

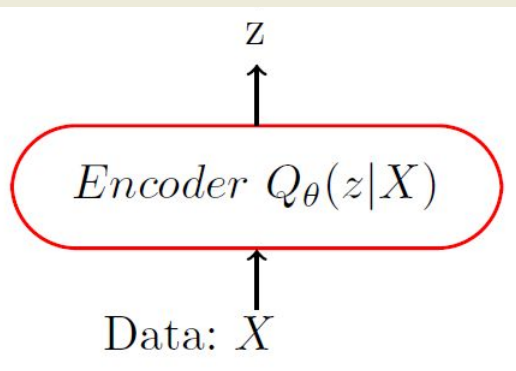
Diagnostic Breakdown: Standard AE vs. VAE

Dimension	Standard AE	Variational Autoencoder (VAE)
Latent Representation	Deterministic Point ($\mathbf{h}$)	Gaussian Distribution $\mathcal{N}(\mathbf{0}, \mathbf{I})$
Encoder Output	Hidden vector $\mathbf{h}$	Parameters μ and Σ
Decoder Input	Fixed point $\mathbf{h}$	Stochastic sampled variable $\mathbf{z}$
Primary Goal	Denoising / Compression	Generative Modeling

VAE: neural network perspective

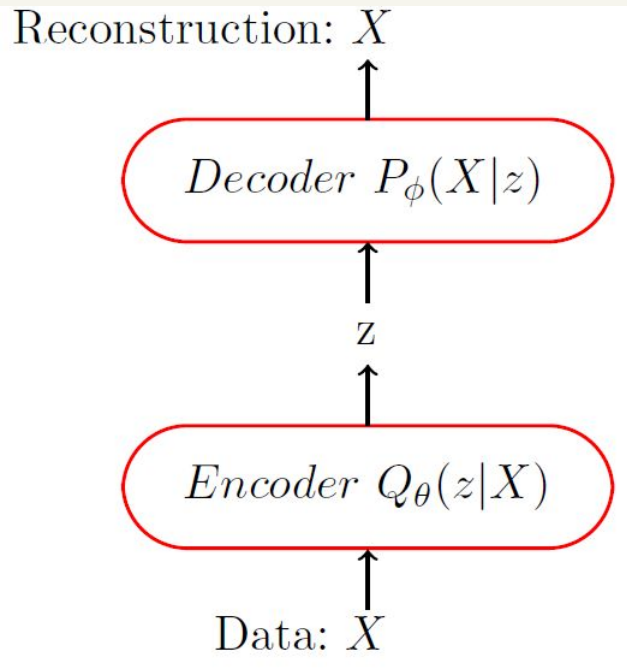
Goal 1

- Learn a distribution over latent variables $Q(z|X)$.



θ : The parameters of the encoder

VAE: neural network perspective



Goal 1

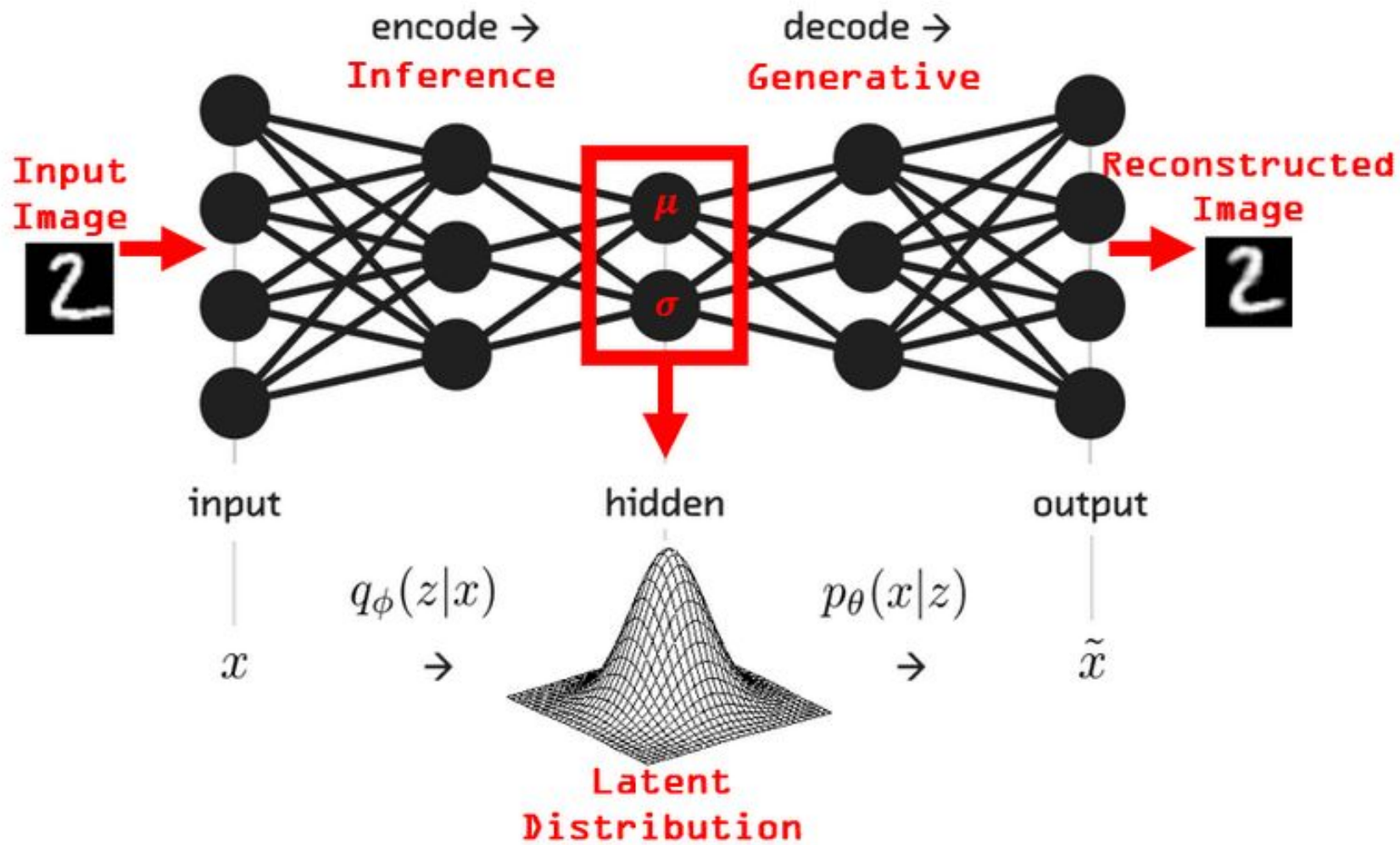
- Learn a distribution over latent variables $Q(z|X)$.

Goal 2

- Learn a distribution over the visible variables $P(X|z)$.

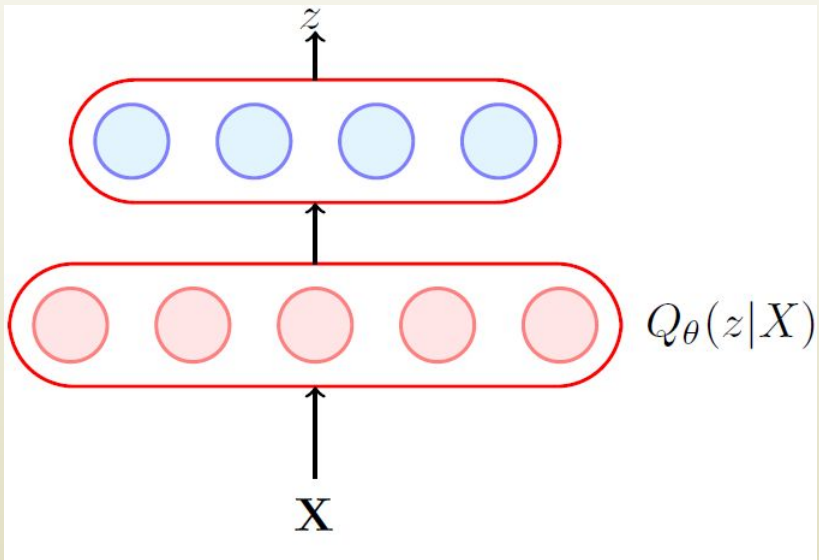
θ : The parameters of the encoder

ϕ : The parameters of the decoder



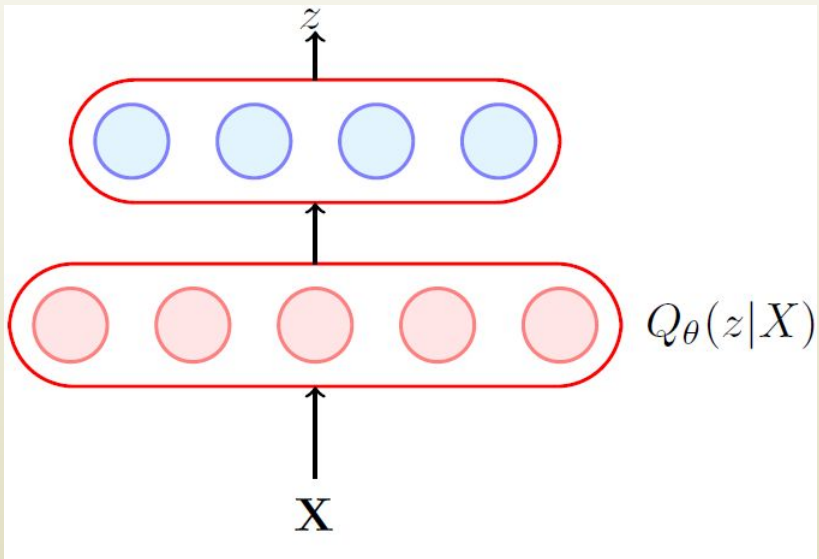
The encoder

What is the meaning of learning a distribution?



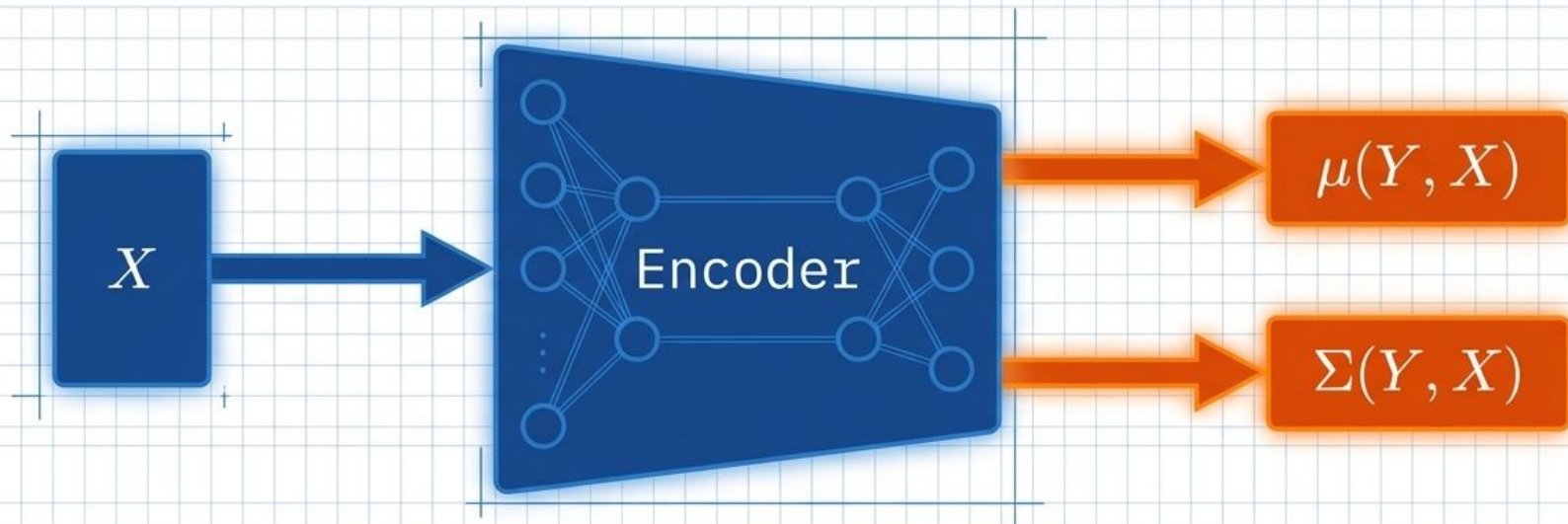
The encoder

What is the meaning of learning a distribution?



Learning the parameters of the distribution.

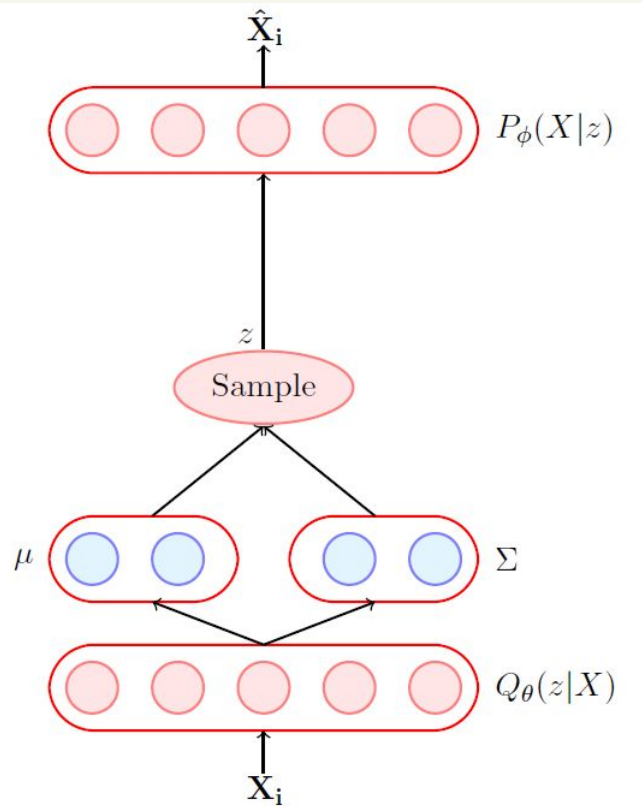
The Neural Network Perspective: The Encoder



Mechanics: The VAE encoder does not output a latent vector z .

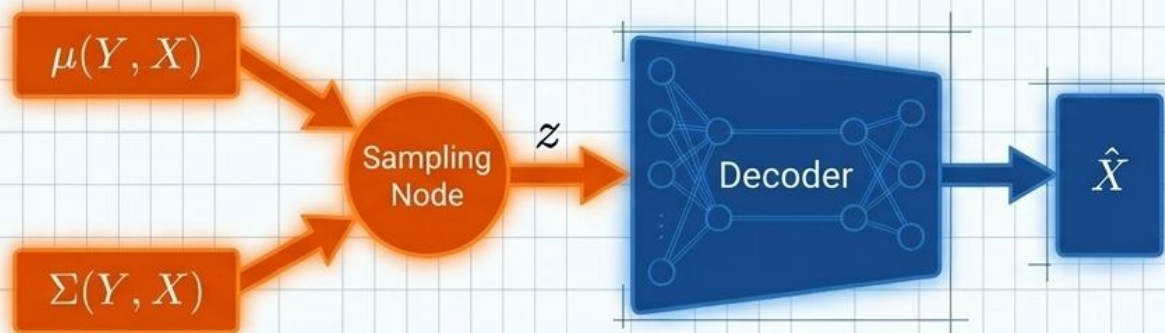
Output: It outputs the parameters of a probability distribution $Q_{\theta}(z|X)$. We assume this is a multivariate Gaussian distribution.

The decoder



The job of the decoder is to predict the probability distribution over X : $P(X|z)$

The Neural Network Perspective: The Decoder



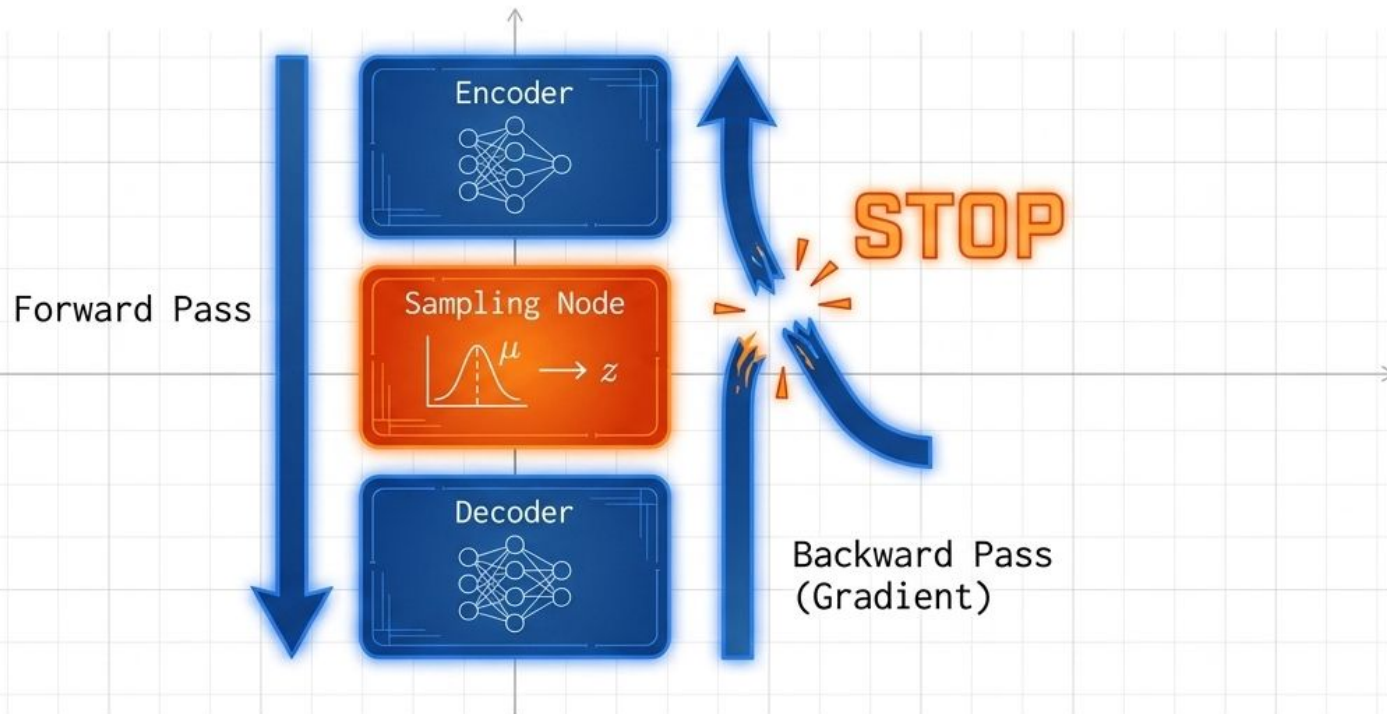
The Stochastic Step

Because the latent space is a distribution, the Decoder must reconstruct the input from a noisy sample z .

The Effect

This forces the decoder to be robust to variance and ensures smooth, continuous transitions across the latent space.

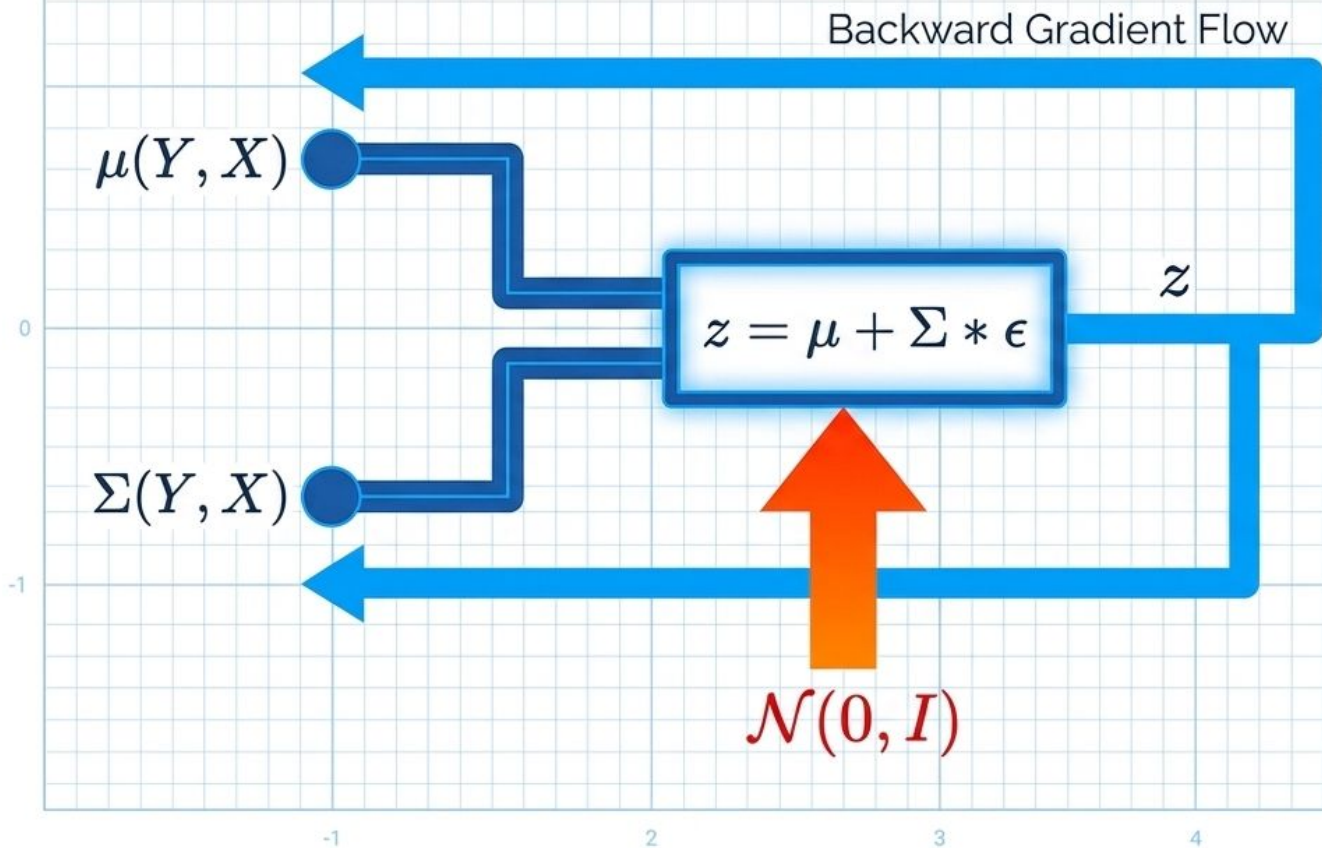
The Training Roadblock: The Sampling Problem



The Issue: Neural networks train via Gradient Descent and backpropagation.

The Trap: The sampling operation $z \sim \mathcal{N}(\mu, \Sigma)$ is random. You cannot backpropagate gradients through a stochastic node. The network is mathematically broken.

The Solution: The Reparameterization Trick



The Fix: We move the randomness to an auxiliary input layer (ϵ).

Result: z is now a deterministic function of μ , Σ , and standard normal noise. The model is now end-to-end differentiable.

Equation Deconstruction: The VAE Loss Function (ELBO)

The diagram illustrates the VAE Loss Function (ELBO) equation:
$$\mathbb{E}_{z \sim Q}[\log P(X|z)] + \text{KL}[\mathcal{N}(\mu(Y, X), \Sigma(Y, X)) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})]$$
 The equation is presented in blue and orange. A blue bracket under the first term points to its description, and an orange bracket under the second term points to its description. Shaded regions and arrows highlight specific parts of the equation.

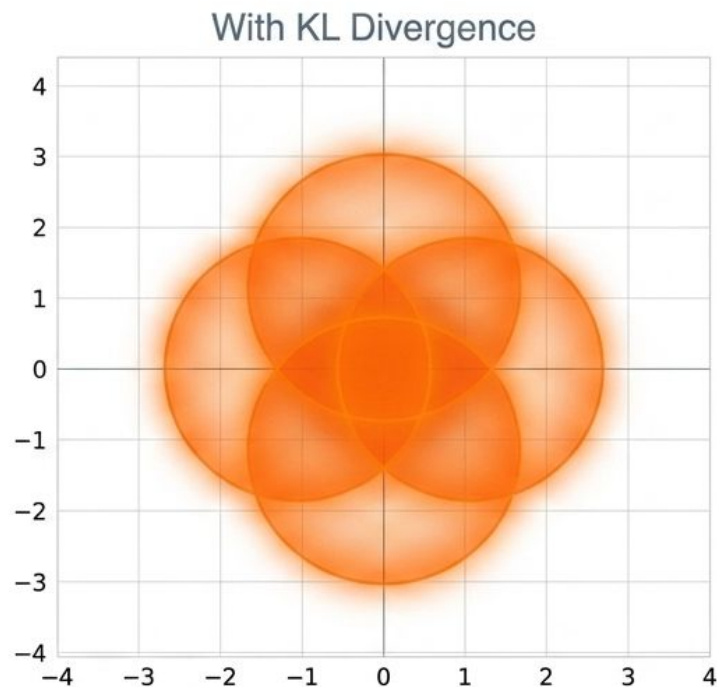
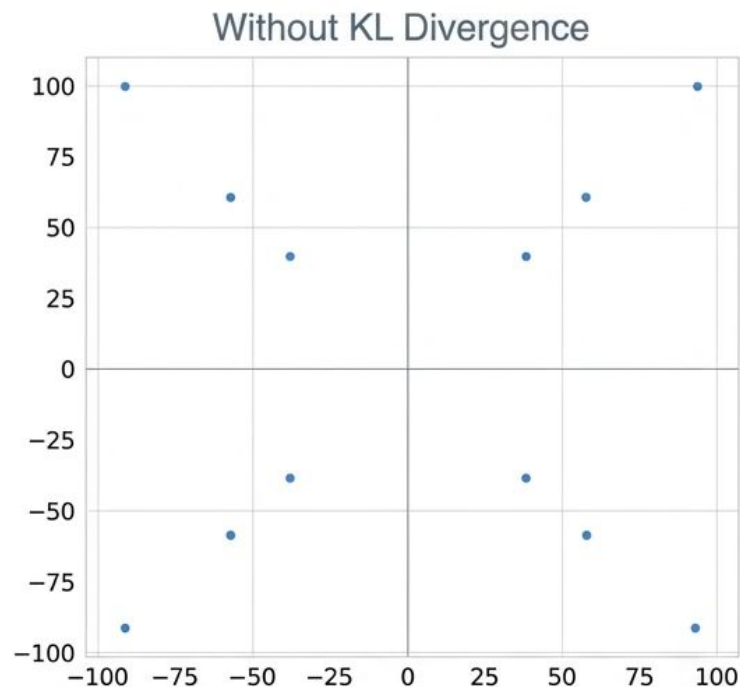
Reconstruction Loss

Term 1 (Decoder's Job):
Forces the model to accurately
reconstruct the data from
the noisy sample.

KL Divergence Regularization

Term 2 (Encoder's Constraint): Penalizes
the network if the encoded
distributions deviate from a standard
normal distribution.

Shaping the Space: The Role of KL Divergence



The Cheat: Without KL divergence, the encoder would cheat by shrinking variance to zero and moving clusters infinitely far apart, reverting to a standard [Autoencoder](#).

The Constraint: KL Divergence forces the distributions to heavily overlap and resemble a Standard Normal Distribution, $N(0, I)$. This creates the continuous space required for generation.

The Rosetta Stone: Two Perspectives

Neural Network Terminology

Encoder

Decoder

Loss Function

Probabilistic Graphical Terminology

Inference Network. Approximates the intractable posterior.

$$Q(\mathbf{z}|\mathbf{x})$$

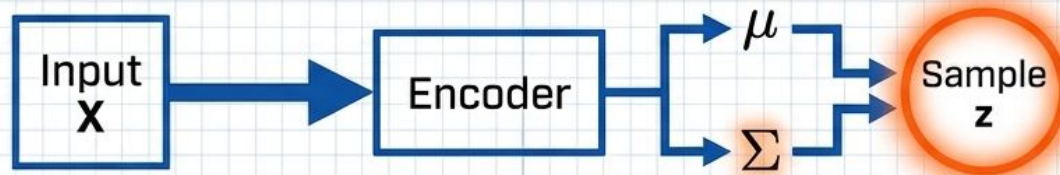
Generative Network. Models the likelihood of data given latents.

$$P(\mathbf{x}|\mathbf{z})$$

Negative ELBO. Maximizing the Evidence Lower Bound.

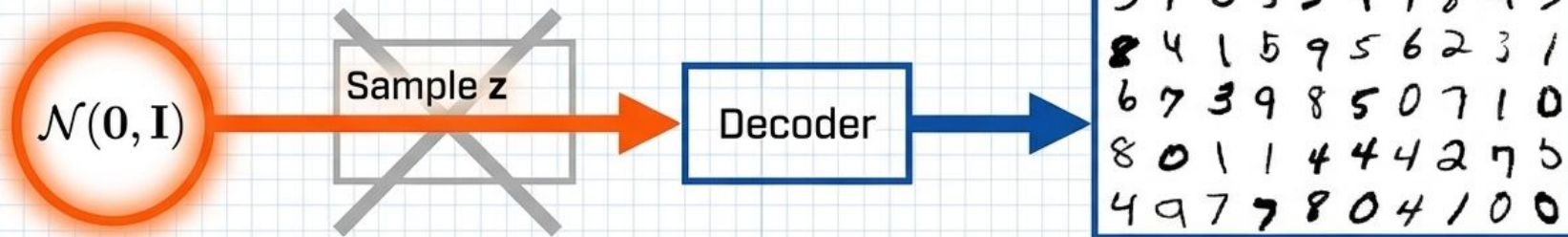
Putting It Together: Test Time Execution

Abstraction



Feed data through the encoder to extract parameterized distributions.

Generation



Because KL divergence forced our latent space to resemble a standard normal distribution, we discard the encoder entirely. We sample pure random noise, feed it to the decoder, and generate novel, plausible data from thin air.